UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
CURSO DE ENGENHARIA DE COMPUTAÇÃO

BRUNO ALMEIDA DA SILVEIRA

B.A.L.A.S: a tool for Benchmarking ANN Latency for ARM-based Systems

Work presented in partial fulfillment
of the requirements for the degree of
Bachelor in Computer Engineering

Advisor: Prof. Dr. Gabriel Luca Nazar

Porto Alegre
November 2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
Reitora: Profª. Marcia Cristina Bernardes Barrosa
Vice-Reitor: Prof. Pedro de Almeida Costa
Pró-Reitor de Graduação: Profª. Nádya Pesce da Silveira
Diretor do Instituto de Informática: Prof. Luciano Paschoal Gaspary
Coordenador do Curso de Engenharia de Computação: Prof. Paulo Francisco Butzen
Bibliotecário-chefe do Instituto de Informática: Alexsander Borges Ribeiro

“I don’t want to believe. I want to know.”

— CARL SAGAN

ACKNOWLEDGEMENTS

First and foremost, I thank my advisor, Dr. Gabriel Nazar, who guided me through this project, and also who incentivized me to not use any LLMs in the writing of this dissertation. Submitting this work knowing that every single word was written by myself feels extremely gratifying.

I would also like to thank the *Rheinland Pfälzische Technische Universität* in Kaiserslautern for the opportunity to study there for one year, where I was first introduced to this research area. Even though none of the text nor code of this project were developed there, this project would have likely not existed if it had not been for the time working as research assistant under Dr. Vladimir Rybalkin. Furthermore, I also thank Dr. Kai Lampka from RPTU, who kindly provided an NXP development board for me as his student, which was the one used for this project.

Finally, I dedicate this acknowledgement to my parents and my girlfriend, whose support was important during this final semester.

ABSTRACT

The use of Artificial Neural Networks (ANNs) in applications with embedded sensors, such as image recognition and signal anomaly detection, is widespread, typically involving data being sent via a network, so that a more powerful machine does the heavy processing. TinyML is a research field that proposes running a compressed version of the neural network on the actual embedded processor, eliminating the network latency and reducing overall power consumption. One of the biggest challenges of TinyML is design space exploration, which can be accelerated with Neural Architecture Search (NAS). The problem with this approach in the context of TinyML is that the latency performance results are often estimated via proxies instead of being measured on the target embedded device, which can lead to suboptimal results. This work proposes the implementation of a benchmarking tool that allows a NAS server to automatically compile, deploy and test inference of a proposed TensorFlow Lite model on real hardware of an ARM Cortex-M microcontroller. This tool was then used to run experiments on a total of 397 representative TinyML models to evaluate the quality of latency estimation proxies, as well as to document the total time of the process. Results show correlations from 0.88 to 0.99 for number of MACs and from 0.75 to 0.99 for workstation CPU inference, as well as demonstrating that it is possible to achieve on-MCU profiling on the order of 20 seconds per model.

Keywords: TinyML. automation. benchmarking. UFRGS.

B.A.L.A.S: uma ferramenta para Benchmarking de Latência de ANNs para Sistemas baseados em ARM

RESUMO

O uso de Redes Neurais Artificiais (do inglês, ANNs) em aplicações com sensores embarcados, como reconhecimento de imagens e detecção de anomalias em sinais, é amplamente difundido, geralmente envolvendo o envio de dados por meio de uma rede para que uma máquina mais potente realize o processamento. TinyML é um campo de pesquisa que propõe a execução de uma versão compactada da rede neural diretamente no processador embarcado, eliminando a latência da rede e reduzindo o consumo de energia. Um dos maiores desafios do TinyML é a exploração do espaço de projeto, que pode ser acelerada por meio de Busca de Arquiteturas Neurais (do inglês, NAS). No entanto, o problema dessa abordagem no contexto do TinyML é que os resultados de desempenho frequentemente são estimados por *proxies* em vez de serem medidos diretamente no dispositivo embarcado alvo, o que pode levar a resultados subótimos. Este trabalho propõe a implementação de uma ferramenta de benchmarking que permite a um servidor NAS compilar, fazer deploy e medir automaticamente a inferência de um modelo proposto em TensorFlow Lite em hardware real de um microcontrolador ARM Cortex-M. Essa ferramenta foi então utilizada para executar experimentos em um total de 397 modelos representativos de TinyML para avaliar a qualidade de *proxies* de estimativa de latência, bem como para documentar o tempo total do processo. Os resultados mostram correlações de 0,88 a 0,99 para o número de MACs (do inglês, Multiply-Accumulate operations) e de 0,75 a 0,99 para inferência em CPU da estação de trabalho, além de demonstrar que é possível se realizar medidas de latência de inferência no hardware alvo na ordem de 20 segundos por modelo.

Palavras-chave: TinyML, Automação, Benchmarking, UFRGS.

LIST OF ABBREVIATIONS AND ACRONYMS

IoT	Internet of Things
ANN	Artificial Neural Network
GPU	Graphics Processing Unit
ML	Machine Learning
MCU	Microcontroller Unit
CPU	Central Processing Unit
FPGA	Field Programmmable Gate Array
ROM	Read-Only Memory
RAM	Random Access Memory
NAS	Neural Architecture Search
MAC	Multiply and Accumulate
TFLM	Tensorflow Lite for Microcontrollers
DNN	Deep Neural Network
FLOP	Floating Point Operation
AI	Artificial Intelligence check if not in intro
HIL	Hardware In the Loop
CLI	Command Line Interface
NN	Neural Network
UART	Universal Asynchronous Receiver/Transmitter
SIMD	Single Instruction Multilpe Data
SRAM	Static RAM
NPU	Neural Processing Unit
IDE	Integrated Development Environment
GUI	Graphic User Interface

LIST OF FIGURES

Figure 2.1 Illustration of how neural network layers are structured (SZE et al., 2017)..17
Figure 2.2 Architecture of MLPerf Tiny Image Classification Model (source: author).23
Figure 2.3 Architecture of MLPerf Tiny Visual Wake Words Model (source: author)..24
Figure 2.4 Architecture of MLPerf Tiny Keyword Spotting Model (source: author).....25
Figure 2.5 Architecture of MLPerf Tiny Anomaly Detection Model (source: author)..25
Figure 2.6 Example of how a code generation inference is invoked (Source: author)...26
Figure 2.7 Example of how an interpreter-based inference is invoked (Source: author)26
Figure 2.8 Example of how operators are added to the OpResolver in TFLM (Source: author).....27
Figure 2.9 Example of how TFLM optimizes memory allocations inside the tensor arena (DAVID et al., 2021).....27

Figure 3.1 Latency estimation model for CNNs on a mobile phone (CAI; ZHU; HAN, 2019).....31
Figure 3.2 DNN architectures with optimized latency for different hardware (CAI; ZHU; HAN, 2019).....31
Figure 3.3 Number of MACs vs real MCU measurements (CAI; ZHU; HAN, 2019)...32

Figure 4.1 General overview of B.A.L.A.S. Source: author.....35
Figure 4.2 Detailed modules and functionality of proposed project. Source: author.....37
Figure 4.3 Diagram of the MCNX947 MCU. Source: (NXP, 2025).....39
Figure 4.4 Interface for the MyModel class (source: author).....42
Figure 4.5 UART configuration window (source: author).....43
Figure 4.6 Timer clock configuration window (source: author).....43
Figure 4.7 Solution to tensor arena underestimation (source: author).....47
Figure 4.8 MCUXpresso clock config view (source: author).....49
Figure 4.9 MCUXpresso clock enable hidden button (source: author).....50

Figure 5.1 Distribution of MACs for Image Classification Model variations (source: author).....52
Figure 5.2 Distribution of MACs for Visual Wake Words Model variations (source: author).....53
Figure 5.3 Distribution of MACs for Keyword Spotting Model variations (source: author).....54
Figure 5.4 Distribution of MACs for Anomaly Detection Model variations (source: author).....54
Figure 5.5 Experimental setup with MCU connected via USB to the workstation (source: author).....55

Figure 6.1 Correlation between number of MACs and measured inference time on Image Classification Model (source: author).....57
Figure 6.2 Correlation between number of MACs and measured inference time on Visual Wake Words Model (source: author).....58
Figure 6.3 Correlation between number of MACs and measured inference time on Keyword Spotting Model (source: author).....58
Figure 6.4 Correlation between number of MACs and measured inference time on Anomaly Detection Model (source: author).....59
Figure 6.5 Correlation between measured workstation CPU inference time and measured inference time on Image Classification Model (source: author).....60

Figure 6.6 Correlation between measured workstation CPU inference time and measured inference time on Visual Wake Words Model (source: author) 60
Figure 6.7 Correlation between measured workstation CPU inference time and measured inference time on Keyword Spotting Model (source: author) 61
Figure 6.8 Correlation between measured workstation CPU inference time and measured inference time on Anomaly Detection Model (source: author) 61
Figure 6.9 Execution times for each step of the automated profiling process (source: author) 63
Figure 6.10 Theoretical execution times for each step of the automated profiling process with optimizations (source: author) 64

LIST OF TABLES

Table 3.1 Summary of related TinyML NAS works 32
Table 3.2 Summary of related TinyML latency benchmarking tools 34

Table 4.1 Pre-processed input sizes for the models (bytes) 40
Table 4.2 Results of stm32tflm tool testing (bytes) 45

Table 5.1 Hyperparameters set for variations of Image Classification Model 51
Table 5.2 Hyperparameters set for variations of Visual Wake Words Model 52
Table 5.3 Hyperparameters set for variations of Keyword Spotting Model 53
Table 5.4 Hyperparameters set for variations of Anomaly Detection Model 53

Table 6.1 Compatibility of the tool on the tested variations 56
Table 6.2 Hyperparameters of the Visual Wake Words variations rejected by stm32tflm 57
Table 6.3 Measured correlation between MACs and real latency 59
Table 6.4 Measured correlation between workstation latency and real MCU latency 62

CONTENTS

1 INTRODUCTION 13
2 BACKGROUND 16
2.1 Artificial Neural Networks 16
2.2 Multiply and Accumulate Operations in DNNs 19
2.3 Quantization 20
2.4 Neural Architecture Search 21
2.5 Benchmarking in TinyML 22
2.5.1 MLPerf Tiny 22
2.5.1.1 Image Classification Model 23
2.5.1.2 Visual Wake Words Model 24
2.5.1.3 Keyword Spotting Model 24
2.5.1.4 Anomaly Detection Model 24
2.6 TinyML Frameworks 25
2.6.1 TensorFlow Lite for Microcontrollers 26
2.6.2 STM32Cube.AI 28
3 RELATED WORK 29
3.1 NAS in TinyML 29
3.2 Tools for model profiling in MCUs 33
4 B.A.L.A.S DESIGN AND IMPLEMENTATION 35
4.1 Proposed Project 35
4.1.1 Project Details 36
4.2 FRDM-MCXN947 Development Board 38
4.3 Implementation 39
4.3.1 Data Preprocessing for Models 39
4.3.2 C++ Project 40
4.3.2.1 TFLM Compilation 41
4.3.2.2 Basic Project Setup 41
4.3.2.3 Generic TFLM Model Inference Class 42
4.3.2.4 UART Configuration 42
4.3.2.5 Timer Configuration 43
4.3.3 Build and Deploy Automation 44
4.3.4 Tensor Arena Estimation 44
4.3.4.1 Fixed very large size 44
4.3.4.2 Trial and error 45
4.3.4.3 STM32 utility tool 45
4.3.5 Custom C++ Code Generator 46
4.3.5.1 Resolver function calls generation 46
4.3.5.2 Model binary generation 46
4.3.5.3 Code injection 46
4.3.6 Model Profiler 47
4.3.6.1 Report 47
4.4 Implementation Challenges 48
4.4.1 TFLM compilation flag 48
4.4.2 UART pins 49
4.4.3 MCUXpresso Clock Config Tool 49
5 EXPERIMENTAL SETUP 51
5.1 Model Variations 51
5.1.1 Image Classification Model Variations 51

5.1.2 Visual Wake Words Model Variations 52
5.1.3 Keyword Spotting Model Variations 52
5.1.4 Anomaly Detection Model Variations 53
5.2 Measurements 54
5.2.1 On-Device Measuring 54
5.2.2 On-CPU Measuring 54
6 RESULTS 56
6.1 Compatibility 56
6.2 MCU Latency vs MACs 57
6.3 MCU Latency vs Workstation Latency 59
6.4 Timing 62
7 CONCLUSION 65
REFERENCES 67

1 INTRODUCTION

The Internet of Things (IoT) is a current trend of technological developments in which increasingly more devices are connected and sharing data among each other. These applications promise to facilitate people's lives by bringing to life concepts such as "Smart Homes", as well as being found in important areas like industry, agriculture and healthcare (MOUHA et al., 2021). In this light, growing economical prospects also arise, as the global IoT market is projected to be worth over 8 trillion dollars by 2030 (AL-SARAWI et al., 2020).

In the context of IoT, whenever an ANN (Arificial Neural Network) must be processed with data collected by sensors, it is typically offloaded via the network to a more powerful computing machine, such as a GPU (Graphics Processing Unit)-powered computer. Therefore, the devices that control the sensors normally only serve to send and receive information, creating a dependency on the network for everything ML (Machine Learning) related (DUTTA; BHARALI, 2021). This motivated the emergence of TinyML, which is the concept of running ML models directly on these tiny devices that are on the edge of the network, representing a promising research field in the ever-growing world of Artificial Intelligence (AI) (ABADADE et al., 2023). One common example that highlights the importance of TinyML are keyword spotting applications, which detect trigger words such as "OK Google" (CHEN; PARADA; HEIGOLD, 2014). For these services, it is important that the keyword detection is processed locally in the mobile device, and only after the trigger word is detected, the audio starts being streamed to a server.

Running ML inference on the edge devices instead of a centralized server can bring lots of advantages in some use-cases. In terms of power consumption, the energy associated with computing inference on the small edge node can be much lower than the energy used to transmit the input data wirelessly (BANBURY et al., 2021). On the same note, running a compressed version of an ML model in a small MCU (Microcontroller Unit) instead of a GPU consumes on the order of 1000 times less energy (SAHA; SANDHA; SRIVASTAVA, 2022).

TinyML systems also have the benefit of being independent of network connectivity, since all the necessary computation is executed locally. This is a big factor in applications such as healthcare monitoring wearables, which could fail to detect an emergency if the user is at a remote location without internet. For instance, the authors in (FARAONE; DELGADO-GONZALO, 2020) developed and deployed a model that de-

tects cardiac arrhythmia from electrocardiogram (ECG) signals and runs on a low-power microcontroller with suitable accuracy. Being independent of the internet also means that the latency of sending and receiving the input data via the network is skipped, which can be more adequate for real-time systems (SAHA; SANDHA; SRIVASTAVA, 2022). To illustrate this, we can compare usual latency ranges of cloud computing and TinyML, which are, respectively, 10-500 ms and 0.18-300 ms (ABADADE et al., 2023). At last, security and privacy concerns are also reduced in TinyML systems. In a typical cloud computing setup, the internet-connected server might be vulnerable to attacks that would expose sensitive data, whereas with TinyML, the data never leaves the edge node and is, thus, more protected from attackers (SAHA; SANDHA; SRIVASTAVA, 2022).

When it comes to the actual device on which the small ML models will run, standard research focus was on mobile phones, like in (TAN et al., 2019), but recently there has been a shift towards including also low-power microcontrollers (BANBURY et al., 2021). Compared to mobile CPUs (Central Processing Units), MCUs consume on average 40 times less power (SAHA; SANDHA; SRIVASTAVA, 2022) and are very cheap, costing on the order of a single dollar (SAHA; SANDHA; SRIVASTAVA, 2022). Another advantage of MCUs in contrast to, for example, FPGAs (Field Programmable Gate Arrays), is that they are easily programmable and reconfigurable (PEREGO et al., 2022). This is reflected in the review made in (HAN; SIEBERT, 2022), where they counted 21 published TinyML papers that used microcontrollers, of which most of them were specifically ARM Cortex-M microcontrollers, and only 2 that used FPGAs. Furthermore, FPGAs are also significantly more expensive than MCUs, meaning that the latter can be more appropriate in cost-critical projects (GHANATHE, 2025).

In order to deploy a neural network to a microcontroller, the model weights must fit in the ROM (Read-Only Memory) - usually Flash - together with the code that implements the operations, and the intermediate results of calculations in between layers needs to fit in the RAM (Random Access Memory) (LIBERIS; DUDZIAK; LANE, 2021). Thus, it is challenging to port common ML models to fit in MCUs, since an average microcontroller has around 1 MB of ROM and 128 kB of RAM (SAHA; SANDHA; SRIVASTAVA, 2022), whereas the more powerful devices targeted by the models have gigabytes of ROM and RAM (BURRELLO et al., 2021).

Due to the difficulty of running existing models in MCUs, it becomes necessary to create new model architectures that are deployable in these constrained environments (PEREGO et al., 2022). Hand-crafting these models is a possibility, but because of the

time it takes, it is often replaced by Neural Architecture Search (NAS), an evolutionary strategy that automates the process of design space exploration, iteratively proposing models and testing their performance (SAHA; SANDHA; SRIVASTAVA, 2022). The problem with these NAS approaches is that they usually don't measure inference time on-device, but instead use heuristics such as number of Multiply and Accumulate (MACs) operations, like in (LIBERIS; DUDZIAK; LANE, 2021) and (BURRELLO et al., 2023), or other auxiliary predictive ML models (PEREGO et al., 2022), claiming that measuring on-device would be too time-consuming. However, it is briefly shown in (TAN et al., 2019) that these metrics can be inaccurate, and that the most accurate estimate of inference latency is always executing and measuring on-device.

Another aspect that makes the development of TinyML applications harder is the heterogeneity present not only on the kinds of models and layers that are available to be deployed, but also on the hardware and software that can be used. TinyML frameworks, which are sets of software tools and libraries that help with model deployment on tiny devices, often only support a small subset of CPU architectures, devices and ML operators, which in turn complicates their seamless use in automated development pipelines (ABADADE et al., 2023). As an example, even the most popular (HAN; SIEBERT, 2022) and most platform-agnostic framework, TensorFlow Lite for Microcontrollers, still requires some manual tweaking of parameters and a few lines of custom code to work optimally (DAVID et al., 2021).

In this light, this work proposes the development of B.A.L.A.S, an extensible tool that will help with profiling ANNs on specific MCUs. This tool provides easy-to-use command line arguments to, given a quantized model, generate an optimized C++ project which implements it with TensorFlow Lite for Microcontrollers, compile and deploy the project to the target board and measure inference latency on-device. The goal of this tool is to provide a simple API for a user who wishes to use more accurate real-world measurements as feedback for a Neural Architecture Search, or to speed-up the profiling process for a user who is hand-crafting a model.

Furthermore, this work also proposes using B.A.L.A.S to run experiments with the objective of comparing latency estimation heuristics with the real on-device measurements, to justify or not the extra time used by the real measurements. Finally, this work also aims to fill a research gap and document how long the entire B.A.L.A.S process takes, in order to serve as a reference to aid other TinyML researchers in the decision of wether real on-device latency profiling takes too long or is worth the time investment.

2 BACKGROUND

This chapter discusses and explains some of the key concepts that are relevant to understanding this work. Sections 2.1, 2.2 and 2.3 talk about topics related to how Neural Networks work with the lens of TinyML: Neural Networks definition, multiply and accumulate operations, and quantization. Section 2.4 briefly explains what is Neural Architecture Search and its nuances when working with tiny devices. Finally, Sections 2.5 and 2.6 talk about the software part of TinyML, respectively, about benchmarking standards and TinyML frameworks.

2.1 Artificial Neural Networks

An Artificial Neural Network is a computational method for solving problems that takes inspiration from the human brain. The core idea behind Neural Networks is combining many small units of simple computation, called neurons, in a structured manner where the outputs of a layer are used as the inputs of the next, so that together they can make complex computations. Specifically, the networks with many layers (with multiple hidden layers) are known as Deep Neural Networks. DNNs are especially good at classification tasks, which is why over the last years their application has exploded in various fields, such as image and speech recognition (SZE et al., 2017).

The key element in a neural network is the neuron, and it is essential to understand how it operates. A neuron takes N inputs and has one output, which is computed by Equation 2.1, where $y \in \mathbb{R}$ is the output, $x \in \mathbb{R}^N$ is the vector of N inputs, $w \in \mathbb{R}^N$ is a vector of N weights, and $b \in \mathbb{R}$ is the bias of the neuron (SZE et al., 2017).

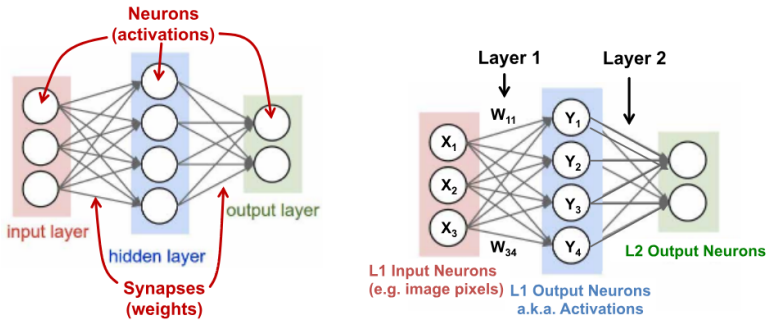
$$y = f(\sum_{i}^{N} w_i x_i + b) \tag{2.1}$$

This "multiply and add" computation can be seen as simply the dot product of vectors x and w. Due to the fact that the dot product is a linear operation, the non-linear function f is added to introduce non-linearity to the system. Since the output of a neuron is also called an activation, this function f is called an activation function (SZE et al., 2017).

In a typical neural network, the neurons are organized in layers, as shown in Figure 2.1. A layer can be seen as the connections between two groups of neurons and their

collective output. To illustrate this, let us take Figure 2.1 as an example. Layer 1 connects the 3 input neurons (which do not perform any computation, as they are simply the input vector of the network) with 4 neurons in the middle. Since each pair of neurons has a connection, there is a weight w_{ij} associated with each pair. The computation in Equation 2.1 is performed 4 times, each with its own bias and weight vector, but on the same input vector, which means that there are 12 weights and 4 biases associated with this layer, as well as 4 activations. These weights and biases are the parameters of a network. They are learned in the training phase and represent a trained instance of a model (SZE et al., 2017). From here on, the terms *model* and *network* will be used interchangeably to refer to an architecture of layers with or without accompanying parameters. A matrix of parameters, such as a matrix of weights, an array of biases, and even an array of activations, can also be called a tensor.

Figure 2.1: Illustration of how neural network layers are structured (SZE et al., 2017)



(a) (b)

Developing applications that use DNNs consists of two main phases: training and inference. The training phase is when a dataset of possible input vectors, each with their respective labels, is fed through the network in an iterative process during which the network's parameters are adjusted following some optimization algorithm like Stochastic Gradient Descent. On the other hand, the inference phase is when the model is already trained and must execute its computation on new unseen data, which is what it constantly does after being deployed to an application (SZE et al., 2017). Since this work focuses on the inference phase, exploring training techniques is outside our scope, and further sections will be restricted mainly to topics that concern inference.

Even though the example in Figure 2.1 shows all pairs of neurons connected in a layer, there are other types of layers (also called operators) that are commonly used in neural networks. In that particular case, it is known as a Fully Connected layer, which can also be represented as the matrix multiplication shown in Equation 2.2, where $y \in \mathbb{R}^M$ is

the vector of M outputs, $x \in \mathbb{R}^{\mathbb{N}}$ is the vector of N inputs, $W \in \mathbb{R}^{M \times N}$ is the $M \times N$ matrix with the weights, $b \in \mathbb{R}^{\mathbb{M}}$ is the vector of M biases, and f is the aforementioned activation function:

$$y = f(Wx + b) \tag{2.2}$$

Other than fully connected layers, there are many other operators that are used in neural networks. Of these, the most relevant are (SZE et al., 2017):

- **Convolution:** Performs a discrete convolution operation between a kernel and the data. The kernel's weights are learnable parameters, and multiple kernels can be convolved to the same data, splitting the network into different parallel channels. Each kernel extends through the entire depth of the channels. The stride parameter determines the length of the "jump" the kernel performs while convolving, so that a higher stride results in lower resolution on the following layer;
- **Depthwise Convolution:** Same as the Convolution, but each kernel only applies to one channel;
- **Pooling:** Reduces the size of the data by grouping neighbor neurons and outputting just the largest (MaxPool), smallest (MinPool) or an average of values (AvgPool);
- **Batch Normalization:** Normalizes outputs of the previous layer so that they have zero mean and unit variance;
- **Add:** Some architectures split the path in two routes, with different layers in each one, which are called Residual Connections (HE et al., 2016). To recombine the split paths, the Add layer is commonly used, which performs elementwise addition (given that they have the same size).

Finally, we cover the activation functions, added at the end of each layer to insert non-linearity. Common activation functions are (SZE et al., 2017):

- **Sigmoid:** $y = \frac{1}{1+e^{-x}}$
- **Hyperbolic Tangent:** $y = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
- **Rectified Linear Unit (ReLU):** $y = max(0, x)$
- **Leaky ReLU:** $y = max(\alpha x, x)$, where α is a small constant
- **Softmax:** Maps values to a valid probability distribution, with all outputs in the range $[0 - 1]$ and with their sum equal to 1. Its formula for the i-th element in a

vector $\mathbf{x}$ is:

$$y_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

2.2 Multiply and Accumulate Operations in DNNs

The number of Mutiply and Accumulate operations (MACs) is a metric used to compare different models in terms of computational complexity (TAN et al., 2019). It serves the same purpose as FLOPs (Floating Point Operations), but does not consider calculations such as the activation functions. Instead, it counts only the multiply and accumulate operations, which are the vast majority of operations computed by a typical DNN. Among the commonly used neural network layers, this is how the number of MACs is computed for each one that performs them (where W_i is the width of the input to the layer, W_o the width of the output, H_i the height of the input, H_o the height of the output, C_i the number of channels of the input, C_o the number of channels of the output, and K the kernel size, assuming a square kernel) (GUO et al., 2019):

- **Fully Connected:** As explained in the previous section, the Fully Connected layer is mostly one large matrix multiplication, which is where the MACs are. For this layer, it is assumed that the input and output data are flat, so only their width matters in the MAC calculation, which is show in Equation 2.3:

$$N_MACS = W_i W_o \tag{2.3}$$

- **Convolution:** For the Convolution Layer, each pixel in an output channel is the result of all the MACs between a kernel and the input on all channels on a given step of the kernel (second term on Equation 2.4). This is done for each pixel in an output channel (first term on the equation) and for each output channel (third term on the equation), so the MAC calculation comes down to Equation 2.4:

$$N_MACS = (W_o H_o)(K^2 C_i) C_o \tag{2.4}$$

- **Depthwise Convolution:** For the Depthwise Convolution Layer it is the same as the regular Convolution, with the difference that, for each pixel on an output channel, it is the result of a 2D kernel multiplication, so the input channels term does not

appear there. Therefore, the MAC calculation comes down to Equation 2.5:

$$N_MACS = (W_o H_o)(K^2)C_o \tag{2.5}$$

2.3 Quantization

Quantization is a technique used for compressing neural networks in which some - or all - parts of it are compressed down to lower bitwidth representations, normally from floating point to integer. This is particularly useful for TinyML, where a small reduction in accuracy in exchange for a large reduction in model size can be desirable, considering that model size is among the main limiting factors for deploying models on devices like MCUs (PEREGO et al., 2022).

There are two alternatives for using quantization: Quantization-Aware Training and Post-Training Quantization. For Quantization-Aware Training, as the name implies, the training phase is altered by injecting simulated quantization operations and other techniques in order to optimize the weights considering exactly how they will be quantized. This tends to deliver better accuracy and to allow for more compression when compared to Post-Training Quantization, but at the cost of needing a custom-made training process. In the common case when a pre-trained model already exists and is not easily re-trainable, or when more flexibility is needed during training, the alternative Post-Training Quantization is used. In it, the model's existing floating-point parameters are directly converted to a smaller floating-point or integer representation. Since this work focuses on the more general case of deploying a pre-trained model to a microcontroller, we will concentrate on Post-Training Quantization (SAHA; SANDHA; SRIVASTAVA, 2022).

Assuming a model has all of its parameters and intermediate values as float32, integer quantization is the one that saves the most computation time when compared to float16, since integer operations are simpler and faster. Quantization can go as far as compressing all weights to one singular bit, but according to the literature review of (SAHA; SANDHA; SRIVASTAVA, 2022), the most frequently used type of quantization among recently published TinyML research is int8, which reduces the storage size in about 4x when going from 32 bits to just 8 for representing model parameters. To perform this conversion on a float32 parameter or input data, Equation 2.6 (KRASTEVA; STOYANOV; JEKOVA, 2025) is used:

$$value_{int8} = \frac{value_{f32}}{scale_factor_{f32}} + zero_point_{int8} \tag{2.6}$$

Similarly, a value can be dequantized using Equation 2.7 (KRASTEVA; STOYANOV; JEKOVA, 2025), which is done, for instance, with the output of a quantized model:

$$value_{f32} = (value_{int8} - zero_point_{int8}).scale_factor_{f32} \tag{2.7}$$

These zero point and scale factor values, known as the quantization parameters, can be determined using different strategies, and different parts of the model can be quantized. In every case, a set of quantization parameters is shared by a tensor. Because of this, these parameters are determined so that the range of possible values that elements of the tensor can take on is fully mapped to the entire range of $[-128, 127]$, in order to have as much precision as possible. With dynamic range quantization, weights are quantized to int8, with their pre-computed scale and zero point values stored as metadata, and, without knowing what range of values the activations can assume, they are dynamically quantized during runtime based on their values during inference. Another strategy, which reduces even more the runtime, is full integer quantization. With it, a representative dataset of possible inputs is given to the quantization engine, upon which it runs inference, evaluates the typical ranges that activations on each layer take, and computes adequate quantization parameters, allowing for everything in the inference phase to be computed using only integer math, no dynamic conversions needed. (ORĂŞAN; SEICULESCU; CALEANU, 2022).

2.4 Neural Architecture Search

Neural Architecture Search (NAS) is the process of automatically searching for neural network architectures, with the goal of optimizing one or more target metrics, with accuracy being a common target. Approaches for defining the search space include layer-wise, cellwise or tree-based. Candidate architectures are proposed and must be evaluated on the target metric. This evaluation either involves training the full model or using a faster proxy metric, such as training with less epochs (TAN et al., 2019) or training on a similar but simpler problem (ZOPH et al., 2018). After a candidate architecture is evaluated, there are different ways to explore the search space in order to find the optimal

architecture, such as evolutionary algorithms, reinforcement learning and gradient-driven (SAHA; SANDHA; SRIVASTAVA, 2022).

For cloud applications, the target metric to optimize is often just accuracy, or a combination of accuracy and latency. In these settings, memory and storage are vast and hardly a concern. However, in TinyML things change, and NAS is crucial for also minimizing ROM and RAM usage of models. Regular NAS frameworks are, therefore, not suitable for TinyML, as their search is configured in a way that will find models that are too large (PEREGO et al., 2022). Because of this, TinyML-targeted NAS frameworks are needed, such as (LIBERIS; DUDZIAK; LANE, 2021), (TAN et al., 2019) and (BURRELLO et al., 2023).

2.5 Benchmarking in TinyML

To supply the demands of TinyML developers, vendors frequently release new software and hardware products, such as frameworks with optimized code or microcontrollers that promise lower power or faster inference for AI (Artificial Intelligence) models. However, without agreed-upon standardized models and measuring methods it becomes difficult to make fair comparisons between products. Benchmarking standards exist to solve this issue, and this section will present what is considered the *de facto* benchmarking suite for TinyML, namely, MLPerf Tiny (BANBURY et al., 2021).

2.5.1 MLPerf Tiny

MLPerf Tiny aims to improve on existing benchmark EEMBC(r)'s CoreMark(r) (GAL-ON; LEVY, 2012), which does not properly represent real ML workloads, and also EEMBC's MLMark(r) (TORELLI; BANGALE, 2021) and MLPerf (REDDI et al., 2020), which, despite considering proper ML workloads, do not offer small enough models to fit in tiny platforms. Another goal of MLPerf Tiny is to be flexible, not making any assumptions about deployment methods nor about TinyML frameworks, in order to support the diversity of this environment.

The benchmark-defined measurement methodology consists of 5 batches of a minimum of 10 inferences or 10 seconds of inferences, averaging the inference times. The same process is defined for energy measurements, which require additional sensors. One

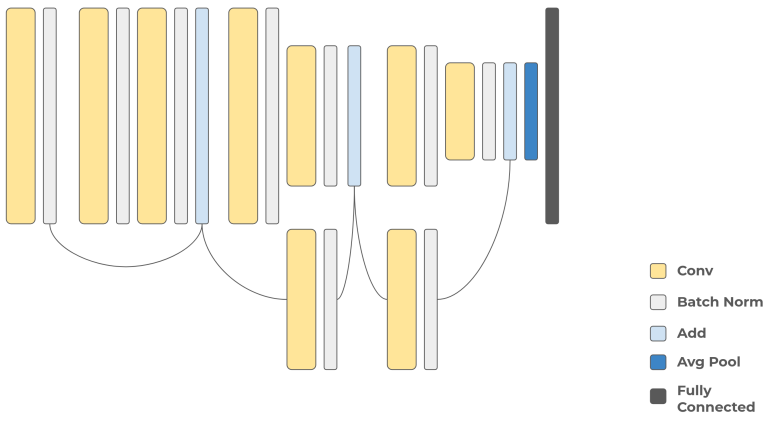
important detail to note is that it measures only the inference call, not including any pre or post-processing such as quantization, dequantization and normalization of inputs. A developer who wishes to benchmark a device must implement functions for reading and writing to UART, which is the interface often used to communicate with the host PC running the other side of the benchmarking suite software, as well as functions for timing the inference, and for initializing the model and running inference, in whatever TinyML framework chosen. A reference implementation of all these functions is provided for an STM32 MCU board and using TensorFlow Lite for Microcontrollers as framework (BANBURY et al., 2021).

Aside from the measuring methodology, the most important standardization of MLPerf Tiny are the benchmarking models. They define four standard models that solve different typical TinyML tasks: Image Classification, Visual Wake Words, Keyword Spotting and Anomaly Detection. Each model's architecture is briefly explained below.

2.5.1.1 Image Classification Model

The Image Classification model (Figure 2.2) is a version of a ResNet (HE et al., 2016) with three blocks. Each block consists of a series of convolutional and batch normalization layers, with a residual connection skipping each block and connecting at the end. It ends with a pooling layer, followed by a fully connected classifier with softmax activation, indicating the probability of each of the 10 image classes.

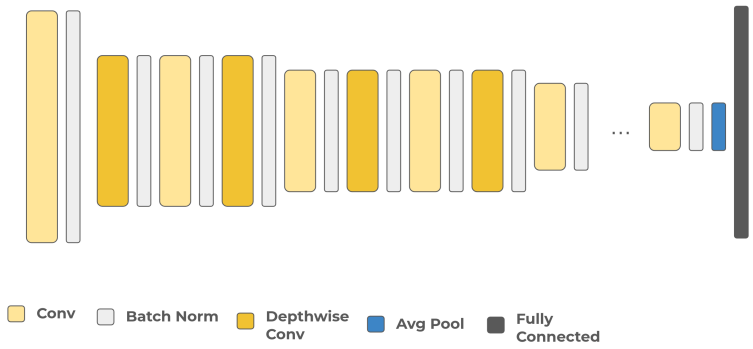
Figure 2.2: Architecture of MLPerf Tiny Image Classification Model (source: author)



2.5.1.2 Visual Wake Words Model

The Visual Wake Words model (Figure 2.3) consists of multiple blocks of alternating convolution and depthwise convolution layers, with batch normalization after each one. After each block, the number of channels increases, and the width and height of the features decreases. It ends with a pooling layer, followed by a fully connected classifier with softmax activation, indicating the probability of wether there is or is not a person in the image.

Figure 2.3: Architecture of MLPerf Tiny Visual Wake Words Model (source: author)



2.5.1.3 Keyword Spotting Model

The Keyword Spotting model (Figure 2.4) initially performs a convolution with a rectangular kernel. It then alternates between depthwise and regular convolution, all with regular square kernels, with batch normalization in between and keeping the same number of channels throughout the network. It ends with pooling and a fully connected classifier with softmax activation, which determines the probabilities of each class of words of the spectrogram given as input.

2.5.1.4 Anomaly Detection Model

The Keyword Spotting model (Figure 2.5) has the simplest architecture among the four benchmarking models. It is an autoencoder, which means that it attempts to compress and decompress the given input signal, and when it fails to reconstruct it properly, it might indicate an anomaly not seen on the training data. It accomplishes this with only fully connected layers, with a central layer that is much narrower than the others, where the compression happens.

Figure 2.4: Architecture of MLPerf Tiny Keyword Spotting Model (source: author)

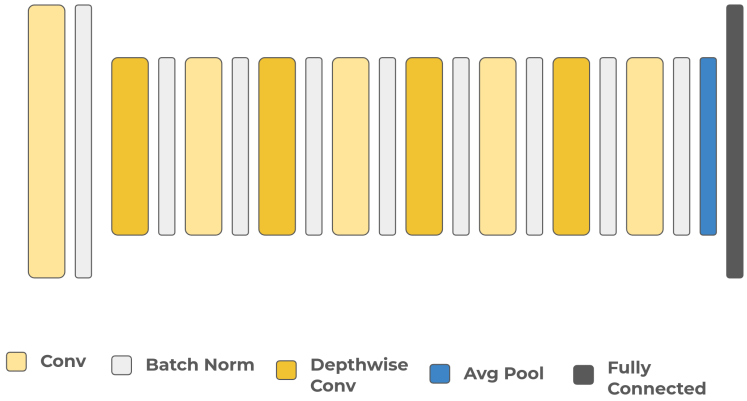
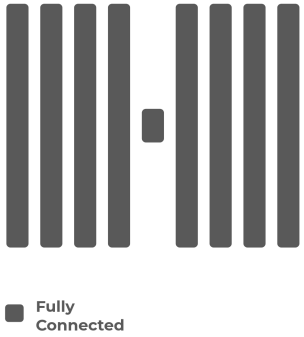


Figure 2.5: Architecture of MLPerf Tiny Anomaly Detection Model (source: author)



2.6 TinyML Frameworks

In order to accomplish the task of running a trained neural network on a microcontroller, the standard way is to use an existing TinyML framework. A framework in this context is a set of tools which provide, among others, two main functionalities: a way to store the models parameters and layer operations in ROM, and also a C/C++ library that implements initialization and inference (SAHA; SANDHA; SRIVASTAVA, 2022).

There are lots of available TinyML frameworks, such as μTVM [1], STM32Cube.AI [2], μTensor [3], DORY (BURRELLO et al., 2021) and TensorFlow Lite for Microcontrollers (DAVID et al., 2021). Since TensorFlow Lite for Microcontrollers (TFLM) was chosen for this work, in this section we will explore in more detail how it works.

[1] https://tvm.apache.org/docs/v0.9.0/topic/microtvm/index.html
[2] https://stm32ai.st.com/stm32-cube-ai/
[3] https://github.com/uTensor/uTensor

2.6.1 TensorFlow Lite for Microcontrollers

TFLM was developed with the aim to be a hardware-agnostic, minimal inference framework. This means that, for the sake of portability, it does not rely on features that lots of low-power embedded processors don't have, like operating systems or some specific hardware integration. The inference library is a set of C++ files that can be compiled to virtually any platform, though they also provide Makefiles for specific popular MCU cores, such as the ARM Cortex-M line. For the tools that generate the model in ROM, the developers reused the mature structure of TensorFlow Lite, which convert a pre-trained .tflite model into a C array.

The developers of TFLM chose to make it interpreter-based, as opposed to directly calling the operator functions. A framework that does direct sequential operator calls looks something like Figure 2.6 - in this case, the operator calls, as well as the initialization of all the proper weights for each operator, must be either written by the developer or generated by a generator provided by the framework's toolchain. On the other hand, in the interpreter case, the model stored in ROM holds not only the weights, but also which operators belong to which layer, and the interpreter parses the model in runtime, deciding which operator to invoke. This adds some overhead, but the authors of (DAVID et al., 2021) claim that the runtime of an inference call is majorly dominated by the operators themselves, and that the interpreter overhead is negligible when compared to them. Therefore, TFLM's inference invocation looks something like Figure 2.7, which adds flexibility, allowing that a different model is used by simply changing which model array is loaded in the interpreter, not having to regenerate and recompile the entire project.

Figure 2.6: Example of how a code generation inference is invoked (Source: author)

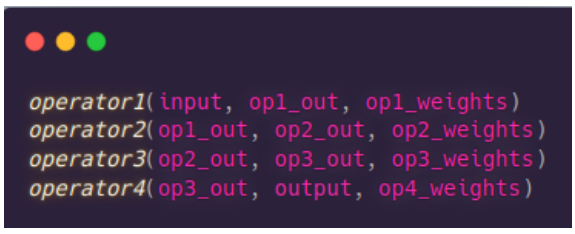
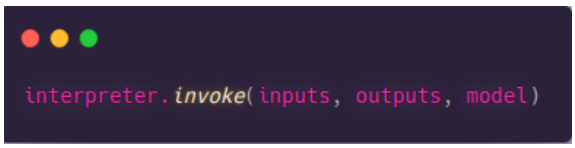


Figure 2.7: Example of how an interpreter-based inference is invoked (Source: author)



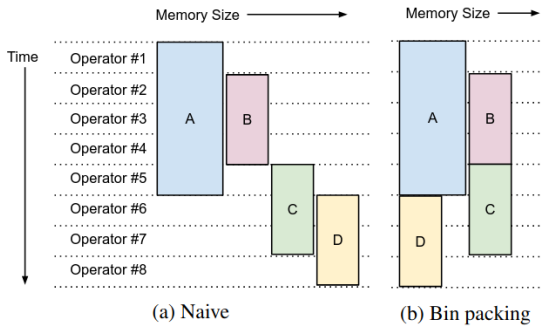
Another element of TFLM is the OpResolver. Since the operators to be called are determined in runtime, TFLM needs a way to know at compile-time what operators will actually be needed, otherwise it would link all the functions of the library and the binary would be much larger than necessary. To solve this issue, TFLM requires that all the used operators are manually added to the resolver object, like in Figure 2.8.

Figure 2.8: Example of how operators are added to the OpResolver in TFLM (Source: author)

```
resolver.AddConv2D()
resolver.AddFullyConnected()
resolver.AddMaxPool()
```

At last, the interpreter also needs a pre-allocated memory region to use for all the activations of the intermediate layers it invokes. This is necessary, since TFLM does not depend on dynamic memory allocation, so the user must choose a size for this array, which is called the tensor arena. Unfortunately, TFLM offers no method to determine a proper tensor arena size, so it falls upon the user to keep testing different values. If the tensor arena size is too small, TFLM throws a runtime error; if it is too large, the inference works but more RAM than necessary is allocated, which is wasteful. Furthermore, it is not trivial to predict how much tensor arena will need to be allocated for a given model, because TFLM uses a memory optimization strategy to reuse tensor arena regions allocated by the interpreter. Figure 2.9 shows an example of how TFLM considers the lifetime of intermediate buffers to minimize the memory needed.

Figure 2.9: Example of how TFLM optimizes memory allocations inside the tensor arena (DAVID et al., 2021)



2.6.2 STM32Cube.AI

STM32Cube.AI [4] is a TinyML framework developed by STMicrocontrollers. Unlike TFLM, it is proprietary and only supports STM boards. It is available via GUI on the STM32Cube IDE or via command line interface, taking a model and generating the entire C++ project that implements the necessary operations to run it. This toolchain also provides tools to help with using TFLM, offering a tool that estimates tensor arena size.

Another big difference between STM32Cube.AI and TFLM is that the former does not make use of an interpreter, rather, it sequentially generates all of the code needed to run inference. Furthermore, it generates code specifically to any supported STM32 microcontroller, making optimizations in the inference code, memory organization and microcontroller configuration. This is shown in (CROCIONI et al., 2020) to have significant savings for STM32 boards, with savings reported of around 20% for inference time, 28% for RAM usage and 14% of ROM usage. It also does not need any interference from the developer, all of the code needed is automatically generated.

[4] https://stm32ai.st.com/stm32-cube-ai/

3 RELATED WORK

This chapter aims to explore the current landscape of research related to benchmarking of ANN latency on tiny devices. Section 3.1 discusses different approaches for Neural Architecture Search optimizing model latency, with examples of works performing real measurements as well as using proxies. Section 3.2 compares tools developed to automate deployment and benchmarking of models specifically on microcontrollers.

3.1 NAS in TinyML

Among recently published literature, some works explore how HIL (Hardware In the Loop) - i.e. using real hardware measurements instead of full simulations - can be used in developing better performing DNNs. Authors in (ESSA et al., 2023) propose a DNN approach for controlling the speed of a Hybrid Electric Vehicle (HEV) instead of the standard Model Predictive Controller algorithm, which is computationally demanding. They propose three different hand-crafted neural network architectures and compare their performance in a simulated environment and also on the real microcontroller. In some key metrics, namely ITAE (Integral Time Absolute Error) and IAE (Integral Absolute Error), it is found that simulations would indicate one architecture performing better than the other, but HIL shows different results, where a slightly underperforming architecture in the simulation would slightly overperform the others when measured on-device. Lastly, the HIL experiments helped determine which of the three proposed architectures was better, which had a performance deemed recommended for use in speed control of HEVs.

Other than testing hand-crafted models on real hardware, a few works have experimented with the idea of integrating HIL and Neural Architecture Search. Authors of (TAN et al., 2019) proposed using real inference latency measurements on Google Pixel 1 smartphones as parameter to be optimized in a multi-objective reward search, justifying that approximation proxies such as number of floating point operations do not yield accurate enough results. In this project, called MnasNet, they produced around 8000 candidate architectures and performed an initial mini-training of just 5 epochs on the ImageNet dataset, in order to save time instead of fully training every candidate. All the partially-trained models were then profiled on-device, compiling using TensorFlow Lite, uploading and measuring inference time. This search, initial training and profiling took 4,5 days, and even though the authors state that 64 TPUv2 devices were used for searching

and training, it is not explicitly said how many smartphones were used, so it is difficult to infer how much time overhead each single on-device measurement added to the process. At the end of the search, the best 15 performing models were fully trained, the fastest of which presented, with similar accuracy, a speed-up of around 2x when compared to other state-of-the-art models running on the same hardware.

Building on top of the ideas of MnasNet, the authors in (CAI; ZHU; HAN, 2019) propose two ways to speed-up the NAS process for CNNs, which, combined, they named ProxylessNAS. They criticize the use of task proxies in NAS, stating that sub-optimal results may be achieved when performing time-saving shortcuts such as training candidate architectures on just a few epochs or initially training on other similar but smaller tasks (e.g., CIFAR-10 instead of ImageNet). To avoid using these task proxies, while also gaining orders of magnitude of speed-up in the search process, they propose the novel idea of training one giant over-parametrized network, which contains all possible building blocks (operators) to be used, and when a candidate architecture is produced, it reuses the weights of those same blocks from the over-parametrized network, saving time by skipping training from scratch thousands of candidate models while using metrics directly for the goal task. With this, they get a search cost of just 200 GPU hours, much less than the claimed 40,000 GPU hours needed for MnasNet, while keeping same latency and accuracy. ProxylessNAS also claims that doing the on-device latency metrics of MnasNet takes too much time and infrastructure, since lots of mobile devices are needed. Instead, they train a model that estimates latency on the target mobile device, needing only one device to do this, and use this model for latency metrics. Figure 3.1 shows the latency times measured on the Google Pixel 1 phone versus the estimated latency of their estimator model, which indicates that the approximation is indeed very close to reality. Another interesting result of this paper is that, when optimizing search considering latency for different target hardware (GPU, CPU, and mobile), the best performing models for each hardware had distinctively different architectures from one another, highlighting how unique different devices are when it comes to running inference, as shown in Figure 3.2.

One important aspect to note is that for both MnasNet and ProxylessNAS, some kind of model deployment workflow was used to test the model on the target device. In the case of MnasNet, this process is unfortunately not detailed, but considering how 8,000 candidate models were deployed on mobile phones, it is reasonable to assume that significant software engineering efforts were put into automating this process. On the other

Figure 3.1: Latency estimation model for CNNs on a mobile phone (CAI; ZHU; HAN, 2019)

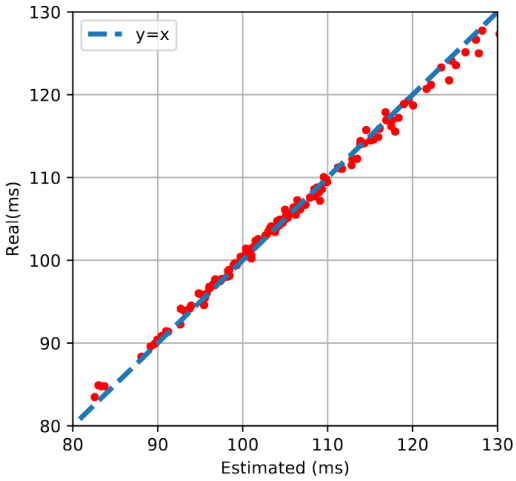
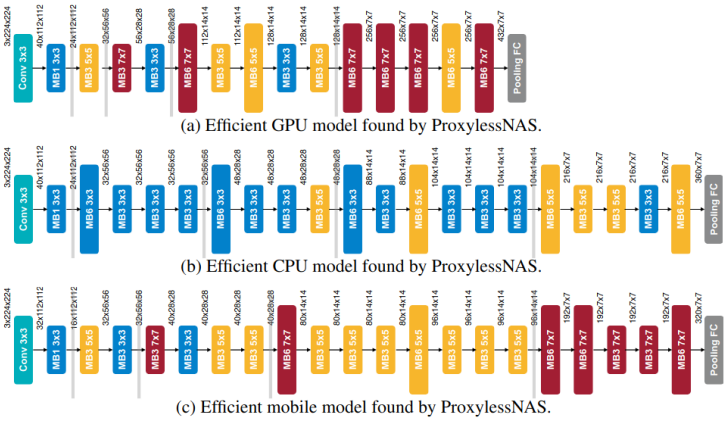


Figure 3.2: DNN architectures with optimized latency for different hardware (CAI; ZHU; HAN, 2019)



hand, even though ProxylessNAS used a latency estimation model instead of deploying all candidate architectures, a number of deployments and latency measurements still had to be made in order to train this estimation model. The lack of details of this process hinders the reproducibility of the experiment. Furthermore, both these works involved automating deployment for mobile phone CPUs, which run on an operating system, non on bare-metal like low-power MCUs, which are the focus of this research. In sum, these two papers show a clear reduced focus on the practical methodology, meaning that other works that seek to reproduce their experiments would need to redevelop these deployment frameworks from scratch.

For an example of NAS with latency as metric to optimize and that targets actual TinyML devices we can cite (LIBERIS; DUDZIAK; LANE, 2021). This work, called

μNAS, proposes a NAS framework designed to properly search the TinyML architecture space, with more granularity, consideration of constraints more proper to microcontrollers and including pruning to produce even smaller models. μNAS considers only fully-int8 quantized models and attempts to reduce the peak RAM usage of the models, as well as the inference latency. However, they do not perform real MCU inference latency measurements, instead opting to use number of MACs as a proxy metric. To justify this choice, they plot the number of MACs vs measured inference time in a NUCLEO-H743ZI2 [1] of 1000 random models from the search space, shown in Figure 3.3, displaying an R^2 index of 0.975. The graph shows a clear correlation, though the less linear section from 0 to 20M MACs is not mentioned by the paper, which could perhaps hinder search quality in that region. μNAS discovered models with up to 13x less peak RAM usage and with the same number of MACs and accuracy as other state-of-the-art models, and models with at least 2x less number of MACs with comparable accuracy.

Figure 3.3: Number of MACs vs real MCU measurements (CAI; ZHU; HAN, 2019)

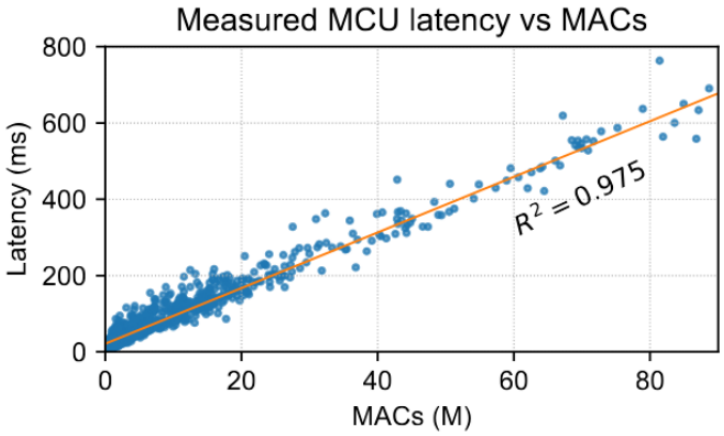


Table 3.1 summarizes the works here presented, and compares them to the proposed tool of this work.

Table 3.1: Summary of related TinyML NAS works

Paper	*Target platform*	*Latency estimation*	*Automated workflow*	*Timing data*
(ESSA et al., 2023)	MCU	Real measurements	No	No
MnasNet	Mobile CPU	Real measurements	Yes	No
ProxylessNAS	Mobile CPU	Predictor model	Yes	No
μNAS	MCU	Number of MACs	Yes	No
B.A.L.A.S	**MCU**	**Real measurements**	**Yes**	**Yes**

Source: author

[1] https://www.st.com/en/evaluation-tools/nucleo-h743zi.html

3.2 Tools for model profiling in MCUs

When it comes to tools developed for automatically deploying and benchmarking machine learning models on microcontrollers, we can cite a few existing works. MLonMCU (KEMPEN et al., 2023) is a python-based MCU benchmarking tool, which makes available a python package and a CLI (Command Line Interface) tool for users. It supports two TinyML frameworks, each with some variations: TFLM either with the standard interpreter or with the TFLMC preprocessor, and TVM with the standard graph executor, the Ahead-Of-Time (AOT) executor or the AOT+ extension, which performs memory planning optimizations. Among the strengths of this work are the fact that it was developed object oriented with extensible interfaces, allowing for extensions from other developers, and also the fact that it was validated successfully on all four MLPerfTiny (BANBURY et al., 2021) standard models. Moreover, MLonMCU is unique in that it shares timing data, with an average build time of 17 seconds among 20 runs of models. We should mention the fact that the total time factor is just briefly discussed, and that it was among only 20 models, as well as that MLonMCU stores the model inputs in ROM instead of sending them in runtime, which takes up ROM that the model weights could be using. Also, there is a lack of fully streamlined build and deployment systems (i.e. the user must always implement their own, there are no pre-developed automations for this).

In the same light, we can look at MLino Bench (BACIU; STIENS; SILVA, 2024), which proposes itself to be a more complete TinyML benchmarking tool. MLino Bench offers a fully automated python-based pipeline with the option to train a model as initial step, or load a pre-trained model, as well as with the option to quantize it. This tool supports neural networks with TFLM as well as classical ML models like Random Forests and Support Vector Machines using a mix of frameworks: emlearn, skearn-porter, embml, micromlgen and m2cgen. After this, the model is automatically deployed to the target device, where it is benchmarked for on-target accuracy, inference time, ROM and RAM usage. The implementation of a fully automated process, from training to deployment, is a big strength of this work; however, we can note that support for the last crucial part of the pipeline, the deployment, is only availabe for Arduino, Atmel SAM, chipKIT PIC32, Teensy and Robotis OpenCR boards.

Another tool that offers benchmarking of TinyML models on MCUs is U-TOE (HUANG et al., 2023). This tool measures RAM, ROM and latency of full models as well as individual operators using the $uTVM$ framework. Its big advantage over other

similar tools is the possibility of remote profiling over TCP (Transmission Control Protocol), though a disadvantage is that the tested model inputs are always randomized, not real inputs sent to the board, which makes it impossible to do any accuracy evaluations. One strength of this work is the availability of fully automated workflows for the tested devices, as well as detailed benchmarking results of the measurements, even though it lacks timing data for build and deploy.

Table 3.2 summarizes the tools here presented, showing clearly the differences between them and B.A.L.A.S.

Table 3.2: Summary of related TinyML latency benchmarking tools

Tool	*Frameworks*	*Automated workflow*	*Timing data*	*Input method*
MLonMCU	TFLM, μTVM	No	Little	ROM
MLinoBench	TFLM, scikit-learn	Yes	No	Serial
U-TOE	μTVM	Yes	No	Randomized
B.A.L.A.S	**TFLM**	**Yes**	**Detailed**	**Serial**

Source: author

In sum, after review of related works, it is clear that measuring on hardware yields the most accurate results for choosing among candidate NN (Neural Network) architectures, even when hand-crafting models. Combining these real deployments and measurements in the NAS loop is a current research field, though not many papers directly explore this, and latency estimators still represent a large amount of this research. Whether it is for hand-crafting architectures, or for integrating with NAS, or for training a latency estimator, the need for a fully automated model deployment and benchmarking for MCUs makes itself clear, and among these benchmarking tools, there is a gap of a fully automated tool that supports a modern ARM Cortex-M microcontroller, as well as a gap for documentation regarding the steps of implementing such a tool and, most importantly, the time it takes to take real measurements of models.

4 B.A.L.A.S DESIGN AND IMPLEMENTATION

This chapter describes the full development process of the B.A.L.A.S tool. Section 4.1 shows an overview of the project, with the higher-level modules and software architecture, Section 4.2 details technical specifications of the MCU used, and Section 4.3 dives deeper with more details of the implementation process, as well as the design decisions that were made along the way. Section 4.4 discusses some of the unexpected challenges of the implementation process.

4.1 Proposed Project

As stated previously, the objective of this work is to develop a tool that, given a pre-trained neural network model, returns a report with inference latency measured on-device. It is meant to be helpful not only for integration with NAS workflows, but also for use in hand-crafting model architectures, so this flexibility guided the design principles. In order to make it suitable for NAS use, it is important to try to minimize the total time the tool takes to run, so that the entire NAS process does not take an unviably long time. Furthermore, it is imperative that the entire process is completely automatic, since any necessary human intervention would render it unusable for NAS. To facilitate its use for hand-crafting models, the results are generated in human-readable format.

Figure 4.1: General overview of B.A.L.A.S. Source: author

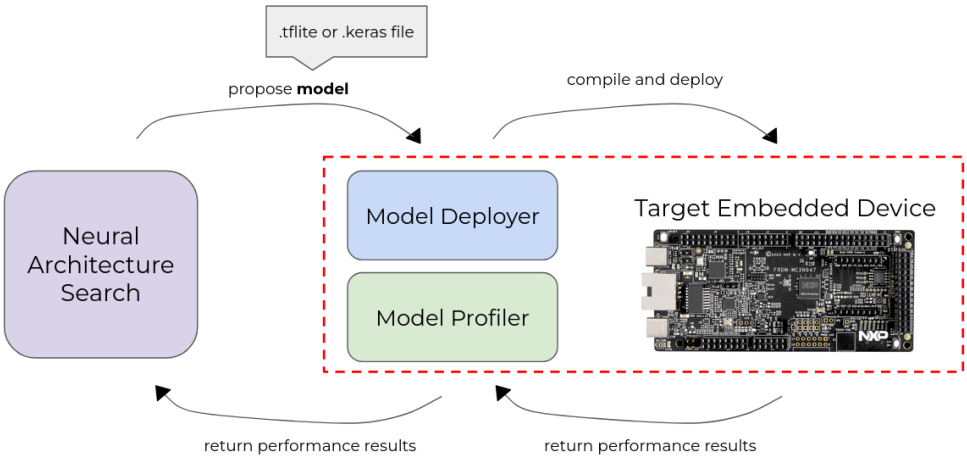


Figure 4.1 shows a typical workflow in which B.A.L.A.S could be used, where the red-dotted box limits clearly what the tool in fact does. In this setup, the proposed tool

runs on a main computer and the target embedded device, which in this case is the FRDM-MCXN947 (details in Section 4.2), is connected to it via USB. This allows not only re-programming of the embedded code, but also communication between the MCU and the main computer. For running inference on the target embedded board, TensorFlow Lite for Microcontrollers was chosen, because it is the most used platform-agnostic TinyML framework (HAN; SIEBERT, 2022), with support for various operators. B.A.L.A.S is executed by invoking a main python script, which integrates a mix of newly-developed and already-existing applications to perform the necessary steps (detailed in Section 4.1.1). Python was chosen for its ease of development and vast libraries for file and communication utilities, as well as for having the TensorFlow Lite library for parsing models. For flexibility, its inputs are passed via command line arguments, allowing for easy integration with automation pipelines and also manual use. The proposed command line options are the following:

- $< model_path >$**:** path to the pre-trained tflite or keras file;
- $< profiling_dataset >$**:** path to the directory containing one file for each f32 (data in the 32-bit floating point format) profiling input tensor;
- $< results >$**:** filename of the results csv output;

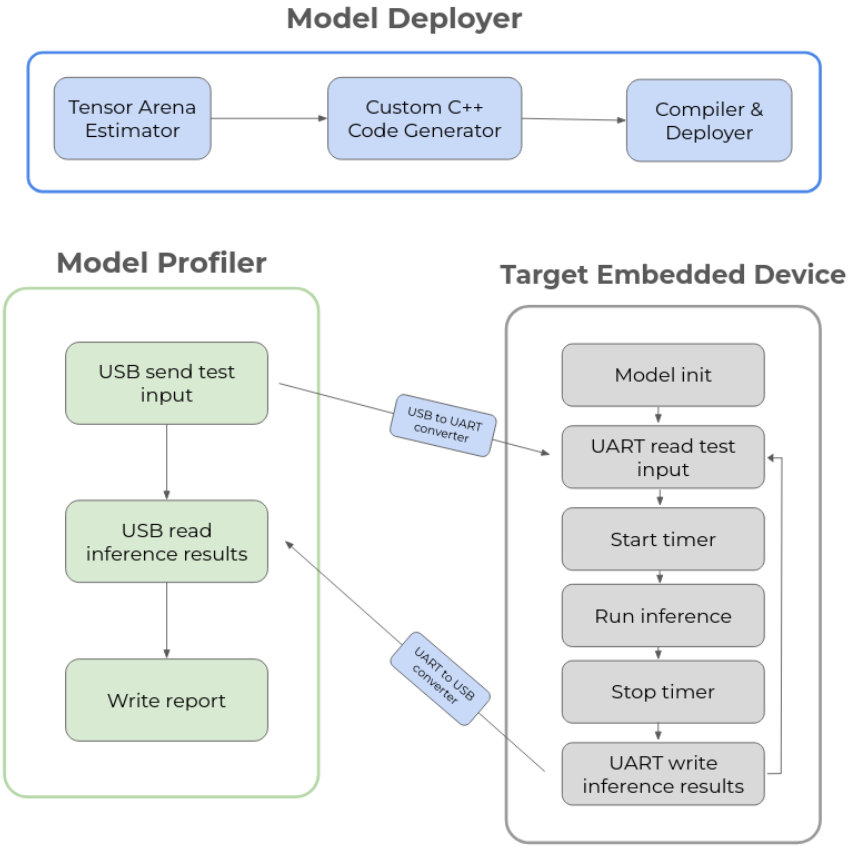
To reduce the scope of the project, it was chosen to only support pre-quantized int8 fully-quantized models. This is justified in the fact that full int8 is the most commonly used type of quantization (as mentioned in Section 2.3), and that for TinyML, most models will need to be quantized to save the limited resources, which if reflected in the fact that MLPerfTiny (BANBURY et al., 2021) had mostly int8 quantized models in its first round of submissions.

4.1.1 Project Details

In order to help understand more clearly the operation of B.A.L.A.S, the diagram in Figure 4.2 shows in more detail the internal modules and functionalities. There are three high-level modules: Model Deployer, Model Profiler and the code developed for the Target Embedded Device. Each one of them will be explained in detail below.

The Model Deployer is the first one to execute. It is responsible for taking a model, performing adequate conversions, generating, compiling and deploying the application that will run inference on the target board. Its modules are explained here:

Figure 4.2: Detailed modules and functionality of proposed project. Source: author



- **Tensor Arena Estimator:** Different tools and methods were analyzed to find the one that can, given a tflite model, determine how much memory must be pre-allocated for the tensor arena in TensorFlow Lite for Microcontrollers. It should avoid unterestimating the value, which causes a runtime error, and overestimate by as little as possible, to avoid using more RAM than necessary;
- **Custom C++ Code Generator:** This python script, using the value of estimated tensor arena size and the model architecture parsed from the tflite file, edits the template C++ project, setting constants and generating layer-specific function calls;
- **Compiler and Deployer:** A python script invokes the proper build and compile methods for the used board.

After the model is deployed to the board, the Model Profiler is activated. It follows a methodology similar to MLPerfTiny (BANBURY et al., 2021), sending, one by one, a batch of test inputs followed by reading the inference time results. As in MLPerfTiny, only the inference is timed, not including quantization, dequantization and any other pre- or postprocessing steps. Then, the process is finished by generating the result reports.

The third part of the project is the C++ template project that implements infer-

ence. Parts of this code are edited by the Model Deployer to make it customized to the input model, and it executes on the MCU in parallel to the Model Profiler, which runs on the host. Its functionality is to act like a UART (Universal Asynchronous Receiver/Transmitter) server, waiting for inputs, running inference, measuring inference time and returning results. This project consists of the following parts:

- **Compilation of TensorFlow Lite for Microcontrollers:** To accelerate the compilation process, the TFLM library is pre-compiled for the ARM Cortex-M33;
- **Model initialization:** Standard boilerplate TFLM initialization code, leaving gaps where model-specific code must be injected by the Deployer;
- **UART read and write:** Code using the MCU's specific BSP (Board Support Package) implements UART initialization, reading and writing;
- **Timer:** Similar to UART, timing functionality is implemented using manufacturer drivers;
- **Inference:** Once the initialization is done, running inference is as simple as calling one function of the library. The input must be quantized before inference, and dequantized after.

4.2 FRDM-MCXN947 Development Board

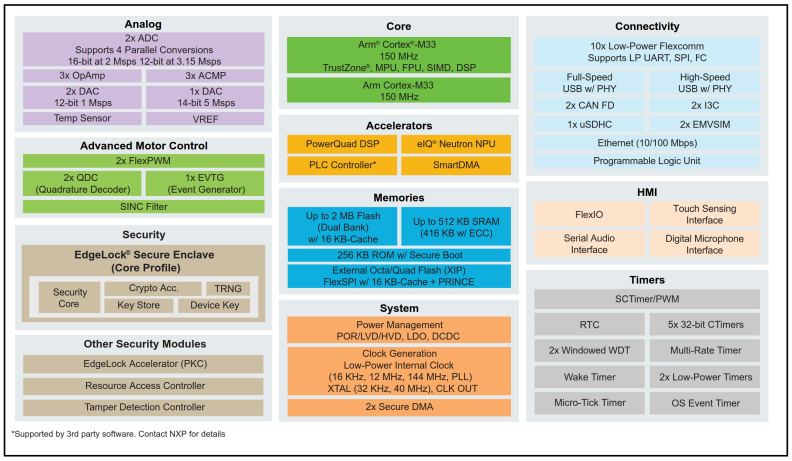
The FRDM-MCXN947 [1] development board is manufactured and sold by NXP Semiconductors. It contains an MCXN947 microcontroller, as well as other utilities for ease of development, such as connectors, converters and easy-to-access pins. Most importantly, it has a USB-C interface, which can be used for powering the board, flashing binaries, debugging with SWD (Serial Wire Debug) and communication with one of the UART modules (NXP, 2025).

The MCXN947 has two ARM Cortex-M33 cores, both with up to 150 MHz clock frequency. One of the cores is the primary core, with extra features like FPU (Floating Point Unit) and support to optimized SIMD (Single Instruction Muliple Data) vector instructions. It has two flash banks of 1MB each, as well as 512 KB of SRAM (416 KB if error correction codes are active). It also has specific hardware accelerators, such as a DSP (Digital Signal Processing) accelerator and an NPU (Neural Processing Unit). However, the scope of this work is limited to the CPU, so these accelerators are not used.

[1] https://www.nxp.com/design/design-center/development-boards-and-designs/FRDM-MCXN947

Figure 4.3 illustrates the hardware modules present on this microcontroller.

Figure 4.3: Diagram of the MCNX947 MCU. Source: (NXP, 2025)



This particular development board was chosen due to the fact that it was designed with TinyML in mind, which is evident by the presence of the NPU for accelerating edge AI. Furthermore, it is an ARM Cortex-M MCU, which is, as previously mentioned, the most widespread in TinyML research.

4.3 Implementation

4.3.1 Data Preprocessing for Models

In order to run the Model Profiler, a small dataset of N float32 input tensors is needed for each of the benchmarking models. Inspired by the methodology of (BANBURY et al., 2021), N=10 was used. Each model uses a different dataset and expects a different kind of preprocessing applied to the input, detailed below:

- **Image Classification:** This model uses the CIFAR-10 dataset, which was downloaded in its entirety. Images of CIFAR-10 are represented as 32x32x3 uint8 values, while the model's quantization parameters map from $[0, 255]$ to int8 range ($[-128, 127]$). In order to maintain a unified interface, considering that all models receive float32 inputs pre-quantization, the 10 selected profiling tensors (one of each class) were first upcasted to float32 before saving to binary files;
- **Visual Wake Words:** This model uses a reduced and downsampled version of the COCO dataset called vw_coco2014_96, with about 100k 96x96x3 images in JPEG format, labeled as person or non-person. The model expects 96x96x3 tensors of

normalized float32 pixel values, in the range $[0.0, 1.0]$. Five images were randomly selected from each label, then decoded from JPEG to float32, then finally normalized and saved to binary files;

- **Keyword Spotting:** This model uses the Speech Commands v2 dataset, which are 1 second long .wav files of spoken short words. As input, the model expects a 49x10 MFCC spectrogram (RAWAT et al., 2023) of the audio clip, so a python script was written to be used to convert the 10 randomly selected .wav profiling inputs to float32 MFCC spectrograms to be saved to binary files;
- **Anomaly Detection:** This model uses the ToyADMOS dataset of machine sounds, but only the ToyCar subset. The entire dataset has tens of gigabytes, so only the ToyCar subset was downloaded, which has 10 second long .wav files of the car sounds. As pre-quantization input, the model expects a 1x640 Mel spectrogram (USTUBIOGLU; USTUBIOGLU; ULUTAS, 2023) of the sound, so a python script was written to convert the 5 selected normal inputs and 5 selected anomaly inputs, with the float32 spectrograms saved as binary files in the profiling dataset.

After this preprocessing stage, each model had its profiling directory ready to be passed to the tool via command line argument. The float32 input sizes for each model can be seen on Table 4.1

Table 4.1: Pre-processed input sizes for the models (bytes)

Model	*Input size*
Image Classification	12288
Visual Wake Words	110592
Keyword Spotting	1960
Anomaly Detection	2560

Source: author

4.3.2 C++ Project

With the goal of receiving input, running inference and measuring its time, a C++ project must be configured and coded, which represents the Target Embedded Device block in Figure 4.2. This project was first developed running the Image Classification Model, and later the specific code related to this model was changed to be replaced by the Custom Code Generator (Section 4.3.5).

4.3.2.1 TFLM Compilation

The first step was to compile TFLM into a .a library file specifically for the Cortex-M33. This pre-compilation is critical, otherwise the library would need to be recompiled every time, adding time to the whole automated profiling process. TFLM offers the option to compile using optimized vector instructions for ARM chips, which speeds-up the inference process. Since this option would always be used in a real scenario where latency must be minimized, this compilation flag was set. Below were the commands used to generate the compiled library, as well as the TFLM and third party library headers:

```
make -f tensorflow/lite/micro/tools/make/Makefile
    TARGET=cortex_m_generic TARGET_ARCH=cortex-m33
    OPTIMIZED_KERNEL_DIR=cmsis_nn microlite
make -f tensorflow/lite/micro/tools/make/Makefile
    TARGET=cortex_m_generic TARGET_ARCH=cortex-m33
    OPTIMIZED_KERNEL_DIR=cmsis_nn list_library_headers
    > tflm_headers.txt
make -f tensorflow/lite/micro/tools/make/Makefile
    TARGET=cortex_m_generic TARGET_ARCH=cortex-m33
    OPTIMIZED_KERNEL_DIR=cmsis_nn list_third_party_headers
    > third_party_headers.txt
```

The final step was writing a python script that takes the lists of header files and reorganizes them in a structure better suited for importing in a C++ project. All of it was packaged in a single "tensorflow" directory, with the compiled .a library and all the headers.

4.3.2.2 Basic Project Setup

To develop the embedded code, MCUXpresso IDE was used [2]. This was the clear choice, because it offers tools for building, flashing and configuring the board, as well as example projects. As a starting point, the example "Hello World" project was used. One important configuration in this step was to increase the default stack and heap sizes described in the linker script in order to accomodate the large amounts of data used by ML models.

[2] https://www.nxp.com/design/design-center/software/development-software/mcuxpresso-software-and-tools-/mcuxpresso-integrated-development-environment-ide:MCUXpresso-IDE

4.3.2.3 Generic TFLM Model Inference Class

The main class in the project is MyModel, which, in its constructor, is responsible for loading the model weights from flash memory and allocating all of the structures used by the framework inside the pre-alllocated tensor arena (Figure 4.4). It also exposes the run inference method, which quantizes the input tensor, computes inference, dequantizes the result and writes it to the output. To facilitate development, ModelInput and ModelOutput classes were created as wrappers to dynamically allocated float32 arrays. Parameters that are model-dependent are the tensor arena size, the number of resolver operators and the list of operators used by the model, as mentioned in Section 2.6.

Figure 4.4: Interface for the MyModel class (source: author)

```
class MyModel {
private:
    const tflite::Model* model;
    tflite::MicroMutableOpResolver<N_OPS> resolver;
    tflite::MicroInterpreter interpreter;
public:
    MyModel();
    void run_inference(ModelInput& input, ModelOutput& output);
    int get_input_size();
    int get_output_size();
};
```

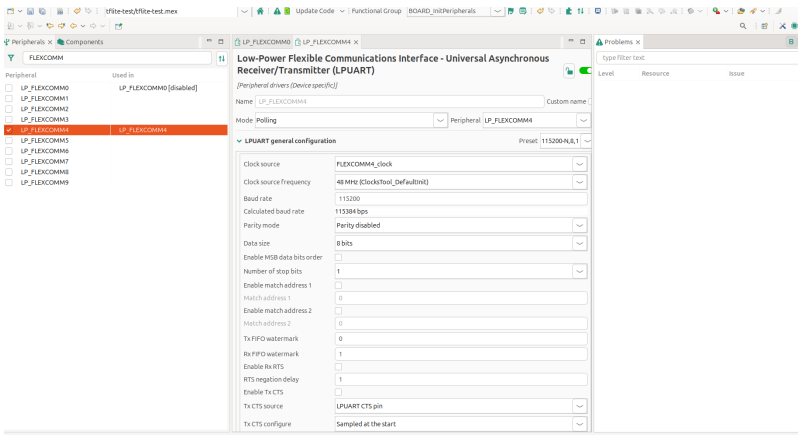
4.3.2.4 UART Configuration

In a microcontroller, the default state is that most (if not all) peripherals are disabled and the developer must write code to manually enable and configure each one on initialization. This is done to save power and only use what is actually needed. In order to save development time, instead of reading among the thousands of datasheet pages and manually setting the correct bits of the memory-mapped registers, the GUI (Graphic User Interface) tools in MCUXpresso IDE (Integrated Development Environment) were used, which generate code that initializes all the peripherals as configured.

For the UART configuration, since the goal is to communicate with the workstation via USB serial interface, it was important that the proper UART module and pins were chosen in such a way that they are the ones connected to the physical USB-C port on the board. Following the Reference Manual [3] for the board, UART peripheral LPFLEXCOMM_4 was used and mapped to pins P1_ 8 and P1_ 9. Figure 4.5 shows the configuration screen with the other UART configurations used.

[3] https://docs.nxp.com/bundle/UM12018/page/topics/related_documentation.html

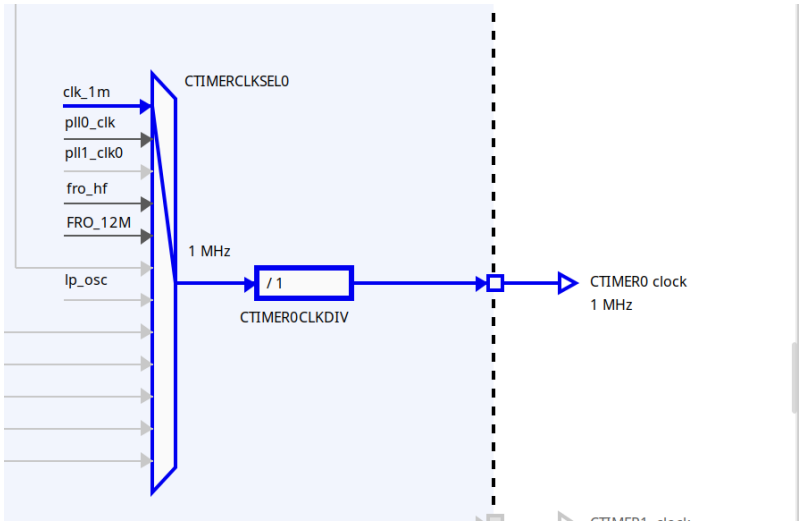
Figure 4.5: UART configuration window (source: author)



4.3.2.5 Timer Configuration

In the same way that UART was configured, a timer peripheral also had to be setup so that it could be used to time the inference. One key difference, though, is that for the timer special attention must be given to the clock routing. Each peripheral can have its clock source chosen, and the timer can be configured to count up every T clock cycles, therefore it is crucial that a proper combination of these two parameters is found. A granularity of microseconds (ABADADE et al., 2023) was needed for the timing of model inference, so, using the clock configuration tool (Figure 4.6), a 1 MHz clock was routed to the timer, and it was set to count up every clock cycle, effectively counting microseconds.

Figure 4.6: Timer clock configuration window (source: author)



4.3.3 Build and Deploy Automation

One of the goals of this tool is to be fully automated, therefore, it cannot be expected of the user to have to open the IDE and interact with the GUI to build and deploy the C++ project. The solution was to write two scripts: one for compiling the project and another for deploying it to the board.

For compilation, it would be possible to manually write a Makefile using the arm-none-eabi cross-compilation toolchain. However, this would take a long time to develop, given the complexity of the project and all the drivers that were added using MCUXpresso IDE. The solution to this problem is in the IDE itself: since it is based on Eclipse IDE, it offers CLI headless build commands which allow compilation of a project in the IDE Workspace with a single command. The build script was then written based on the MCUXpresso IDE Command Line User Guide[4], with special care given to making sure that "make clean" was not executed every time in order to save time by not recompiling files unnecessarily.

The deploy script was also written with help of the IDE. MCUXpresso offers a GUI Flash tool, that identifies NXP boards connected to the USB and allows easy deployment of projects to them. This tool also has an option to display the commands it executes. These commands were then adapted to a script, which is invoked right after compilation.

4.3.4 Tensor Arena Estimation

As explained in Section 2.6.1, TFLM requires a statically allocated buffer of fixed size in which it allocates the structures used during inference. The following options were considered to deal with this:

4.3.4.1 Fixed very large size

Choosing a very large constant tensor arena size (for example: 400 Kb) for all models would work for most of them. The problem is that, if a model actually needs much less than this size, there is significant waste of RAM. For instance, a model that

[4]https://community.nxp.com/pwmxy87654/attachments/pwmxy87654/lpc/53134/1/MCUXpresso_IDE_Command_Line_User_Guide.pdf

needs only 200 Kb of tensor arena, but needs 150 Kb for its input tensor in float32, would not fit in 512 Kb of RAM with this method using a 400 Kb fixed tensor arena size for all models, but it could fit if there was a better estimation for the arena sie.

4.3.4.2 Trial and error

One option could be to start with a small tensor arena size and try to deploy the model: if it does not work, try again with a larger size, and repeat until it works. The problem with this approach is that, while it would find a good arena size, each attempt would involve compiling and deploying to the board, which would take too much time for benchmarking a single model.

4.3.4.3 STM32 utility tool

The tools that come packaged with the framework STM32Cube.AI (Section 2.6.2) also offer utility tools to help using TFLM on STM32 boards. One of these tools, called "stm32tflm", takes as input a .tflite model file and writes to STDOUT how much flash the model takes and how much RAM needs to be allocated in the tensor arena for it.

To evaluate the accuracy of the estimations of stm32tflm, the following was performed for each of the 4 benchmarking models: first, the trial and error method was manually used to find the exact minimum tensor arena size needed for the model; then the STM32 tool was used, and the estimation was compared to the actual smallest arena size found empirically. The results were compiled in the Table 4.2.

Table 4.2: Results of stm32tflm tool testing (bytes)

Model	*Minimum tensor arena size measured*	*stm32tflm estimation*
Image Classification	55000	56832
Visual Wake Words	100730	103933
Keyword Spotting	22825	24576
Anomaly Detection	4325	4096

Source: author

We can see that stm32tflm produces good estimates of the required tensor arena size, with three slight overestimations and one slight underestimation. The biggest overestimation among the four standard benchmarking models is in the Keyword Spotting model, allocating 7.6% more than actually needed. For this work, this is considered acceptable, so stm32tflm was in fact the method chosen for tensor arena size estimation. We

can see, however, that for one of the models it underestimated the size, which causes it to not work. The solution proposed to this problem will be presented in Section 4.3.6.

4.3.5 Custom C++ Code Generator

As mentioned in Section 2.6.1, even though TFLM uses a generic interpreter, some model-specific code is still needed for it to function. This module is responsible for generating and injecting this code in the C++ project in three steps: generate resolver function calls, generate model binary header, and inject code in the project.

4.3.5.1 Resolver function calls generation

To figure out which operators (layers) the model uses, the .tflite file is parsed with the python TFLite library, with which the layer names are extracted. A python dictionary was produced, which maps TFLite layer names to their respective resolver function names. This dictionary was produced by parsing the TFLM micro_mutable_ops_resolver.h header file.

4.3.5.2 Model binary generation

For model initialization in MyModel class, it needs to include a header which contains a C array of the entire binary data of the .tflite file. This is generated with the xxd command line utility tool and edited to have a known variable name.

4.3.5.3 Code injection

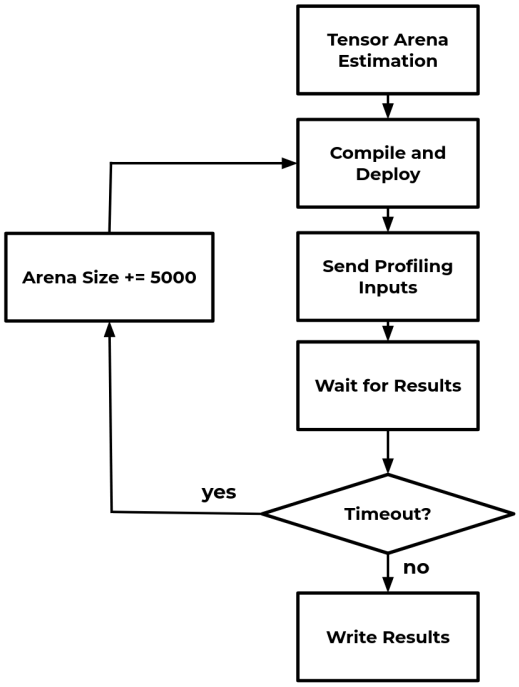
The files that need custom code (model.cpp and model.h) have copies which stay in a templates directory. These template files have filler tags where the tensor arena size, number of resolver operations and resolver function calls should be. In the code injection step, the templates are first copied to the C++ project directory, then the generated values are injected using find and replace. These template copies are needed because, otherwise, after the first execution of the tool, the tags would have been overwritten.

4.3.6 Model Profiler

After the model was deployed to the board and is waiting for test inputs on UART, the model profiler is the module responsible for sending those inputs. It reads all the float32 binary files from the given profiling dataset directory, opens a serial connection to the USB device interface, sends one input tensor, then waits for the MCU's response, which is the inference time in microseconds (as shown in Figure 4.2).

To address the issue of underestimated tensor arena sizes, a timeout of 10 seconds was set on the serial read of the inference timing results. If this timeout occurs, it is assumed that the tensor arena was too small, because the C++ application hangs indefinitely during construction of MyModel if that happens. Then, the tensor arena size is incremented by 5,000, and the flow goes back to compilation (shown in Figure 4.7), where the process is attempted again with this larger arena size.

Figure 4.7: Solution to tensor arena underestimation (source: author)



4.3.6.1 Report

The final output of the tool is a CSV report file, which is opened in append mode to allow results of multiple models to be written to the same file. Each relevant section of the execution of the entire process is also timed and added to the report, with the goal

of producing results that will help analysis of how much time the profiling process took. These are the fields added to the report:

- **model:** Filename of the .tflite model used;
- **arena_size:** Final tensor arena size;
- **macs:** Number of MACs of the model;
- **avg_inference_time_us:** Average time of all the inferences in microseconds;
- **std_inference_us:** Standard deviation of all the inferences;
- **arena_time_ms:** Time taken by stm32tflm tool in milliseconds;
- **mac_time_ms:** Time taken by MAC counting in milliseconds;
- **cpp_codegen_time_ms:** Time taken by C++ Code Generator in milliseconds;
- **compile_time_ms:** Time taken by C++ project compilation in milliseconds;
- **deploy_time_ms:** Time taken by deployment to board in milliseconds;
- **profiling_time_ms:** Time taken to run model profiler in milliseconds;
- **attempts:** Number of attempts needed to get the correct tensor arena size;

4.4 Implementation Challenges

This section aims to document some of the unexpected challenges during the implementation phase. Its main objective is to document these difficulties as well as their solutions, so that other TinyML developers who wish to do similar work do not need to go through them.

4.4.1 TFLM compilation flag

Initially, after the project setup with TFLM compiled and its headers properly organized, the program would crash in the model initialization, even with very simple test models. After a few weeks of trying to solve this blocking issue, looking into TFLM source code, the lack of the build flag “TF_LITE_STATIC_MEMORY” was found to be the source of the problem. Adding it in the IDE project configurations solved the issue.

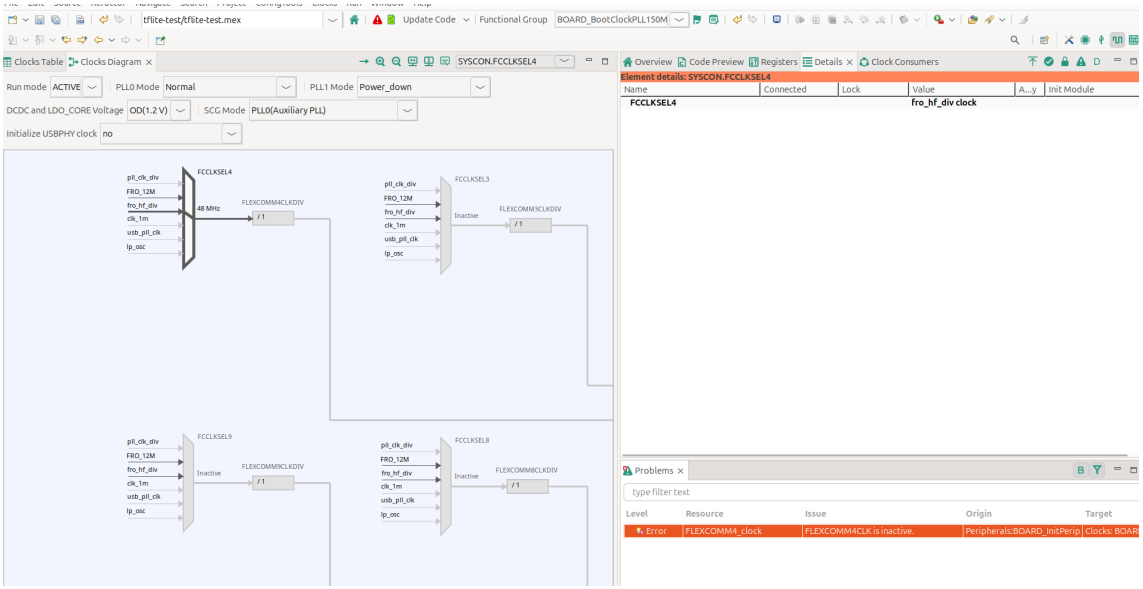
4.4.2 UART pins

Figuring out exactly which pins to configure for UART reads and writes to go through the physical USB-C connector was another challenge. Reading the Reference Manual (mentioned in Section 4.3.2.4), it was initially understood that pins P1_20 and P1_21 had to be used, but this did not in fact output to the USB interface. It was only after a deeper analysis of the Manual that the information of using pins P1_8 and P1_9 was located and sucessfuly tested.

4.4.3 MCUXpresso Clock Config Tool

Another challenge was learning to use the MCUXpresso IDE, which is very complete in terms of features, but has a learning curve for some configurations. The clock configuration tool was particularly unintuitive. Figure 4.8 shows the clock configuration UI, where it is possible to select which clock sources with which multipliers get routed to the peripherals. Without this configuration, neither the timer nor the UART peripherals worked.

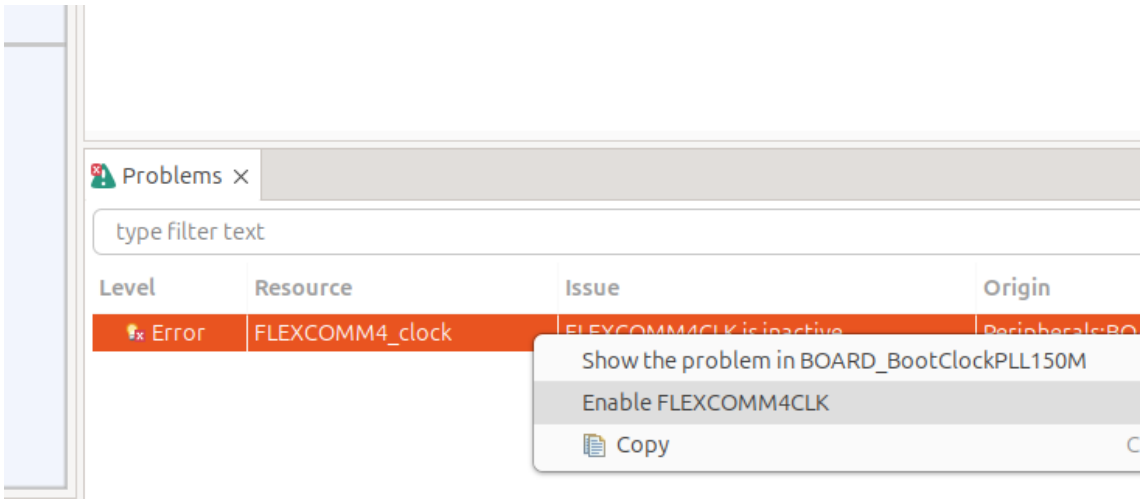
Figure 4.8: MCUXpresso clock config view (source: author)



However, it can be seen in Figure 4.8 that the FLEXCOMM4_clock (for UART) is not enabled, even though the clock signal multiplexer selects a proper clock source. Right-clicking the clock signal did not show any options, and in online tutorials it simply

worked directly. After days of struggling with this problem, the very unintuitive solution was found: the button to enable the clock signal only appeared upon right-clicking the error message, as can be seen in Figure 4.9.

Figure 4.9: MCUXpresso clock enable hidden button (source: author)



5 EXPERIMENTAL SETUP

5.1 Model Variations

In order to evaluate the compatibility of the developed tool, as well as to compare on-device profiling with other model latency evaluation methods, we generated and quantized variations of the four standard MLPerfTiny models. It was not necessary to train each model, since we are only interested on the inference latency, not on the accuracy. A set of possible values for each relevant hyperparameter was assigned, and the variations were generated with all possible combinations of all the pre-defined values of the hyperparameters. The goal was to have between 90 and 110 variations of each model, depending on the number of hyperparameters. The choice of possible hyperparameter values was made to attempt to create wide and continuous distributions of number of MACs, with the goal of having variations with diverse numbers of MACs, as well as to make sure that all models would fit in the ROM of the MCU.

5.1.1 Image Classification Model Variations

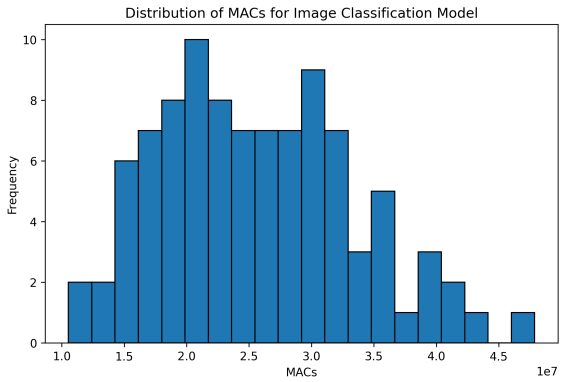
This network is divided in three blocks, each with increasing number of channels. To add more hyperparameters, we added the possibility that each block could have different kernel sizes, instead of fixed at 3. Table 5.1 shows the values used. This produced a total of 96 variations. A histogram showing the distribution of number of MACs generated by the combinations of these hyperparameters can be seen in Figure 5.1.

Table 5.1: Hyperparameters set for variations of Image Classification Model

Hyperparameter	*Values*
CHANNELS_1	16, 24
KERNEL_1	3, 4
CHANNELS_2	32, 48
KERNEL_2	3, 4
CHANNELS_3	64, 96, 128
KERNEL_3	2, 3

Source: author

Figure 5.1: Distribution of MACs for Image Classification Model variations (source: author)



5.1.2 Visual Wake Words Model Variations

The original network is also divided in blocks, multiplying by 2 the number of channels between each block. We made this multiplication factor a hyperparameter, as well as the initial number of channels and kernel size (same for all blocks). Table 5.2 shows the values used. This produced a total of 105 variations. A histogram showing the distribution of number of MACs generated by the combinations of these hyperparameters can be seen in Figure 5.2.

Table 5.2: Hyperparameters set for variations of Visual Wake Words Model

Hyperparameter	*Values*
INITIAL_CHANNELS	4, 5, 6, 7, 8, 9, 10
CHANNELS_MULTIPLIER	1.2, 1.4, 1.6, 1.8, 2.0
KERNEL	2, 3, 4

Source: author

5.1.3 Keyword Spotting Model Variations

This model keeps a fixed number of channels among the blocks, which was chosen as a hyperparameter. It has one initial rectangular kernel and the rest of the convolutions use a fixed square kernel size, both of which were used to make the variations. Table 5.3 shows the values used. This produced a total of 100 variations. A histogram showing the distribution of number of MACs generated by the combinations of these hyperparameters can be seen in Figure 5.3.

Figure 5.2: Distribution of MACs for Visual Wake Words Model variations (source: author)

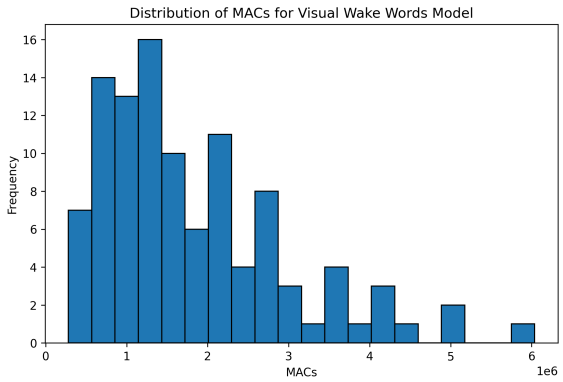


Table 5.3: Hyperparameters set for variations of Keyword Spotting Model

Hyperparameter	*Values*
CHANNELS	32, 48, 64, 96, 128
KERNEL	2, 3, 4, 5, 6
FIRST_KERNEL	5x2, 10x4, 15x4, 20x8

Source: author

5.1.4 Anomaly Detection Model Variations

Being the simplest model, this one would only have 2 possible hyperparameters: size of the encoder/decoder fully connected layers, and size of the central funnelling fully connected layer. We chose to add another intermediate size between these two, with the objective of creating more varied models. Table 5.4 shows the values used. This produced a total of 96 variations. A histogram showing the distribution of number of MACs generated by the combinations of these hyperparameters can be seen in Figure 5.4.

Table 5.4: Hyperparameters set for variations of Anomaly Detection Model

Hyperparameter	*Values*
LARGE_FC	192, 208, 224, 240, 256, 288
MEDIUM_FC	96, 128, 160, 192
SMALL_FC	8, 16, 32, 64

Source: author

Figure 5.3: Distribution of MACs for Keyword Spotting Model variations (source: author)

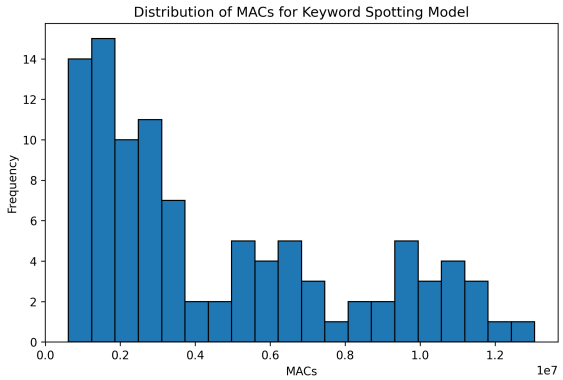
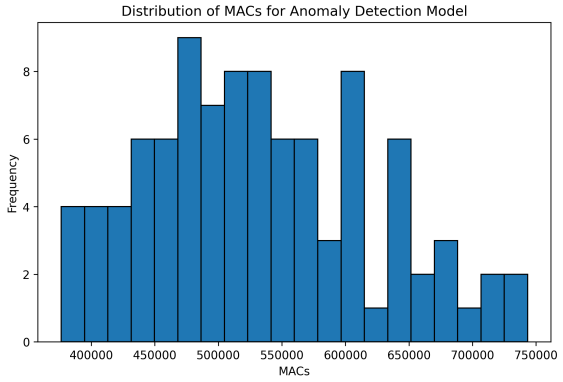


Figure 5.4: Distribution of MACs for Anomaly Detection Model variations (source: author)



5.2 Measurements

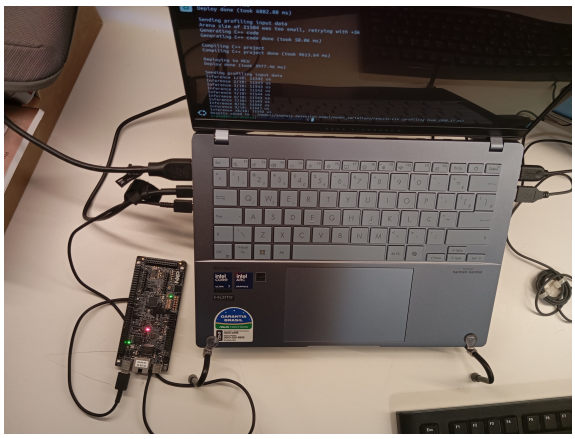
5.2.1 On-Device Measuring

For each of the four base benchmarking models, all of its quantized .tflite variations were passed as input to the profiling tool, with its results appended to the model's results file. The FRDM-MCXN947 development board was connected to the workstation with a USB-C cable. Figure 5.5 shows the experimental setup.

5.2.2 On-CPU Measuring

All the same model variations also had its inference times measured on the workstation CPU, in order to compare this method of estimating inference latency with the real measurement. The workstation was an Ubuntu 24.04.3 LTS laptop with an Intel Core Ultra 7 258V 4.8 GHz CPU with 32 Gb of RAM. This was performed using the

Figure 5.5: Experimental setup with MCU connected via USB to the workstation (source: author)



python TFLite library and process-local timers to ensure that other processes running on the workstation did not interfere with the results.

6 RESULTS

This chapter aims to display and discuss the results of all the proposed experiments. Section 6.1 discusses the compatibility of the tool with different models, while Sections 6.2 and 6.3 compare the real MCU latency measurements with the alternative latency estimation metrics, respectively, number of MACs and inference time on the workstation. Finally, Section 6.4 explores how much time the on-device profiling added to the search process when compared to the other methods.

6.1 Compatibility

To evaluate the compatibility of the tool, we recorded on which of the test models the entire profiling process worked successfully, and on which ones fatal errors occurred. Table 6.1 shows these results.

Table 6.1: Compatibility of the tool on the tested variations

Base Model	*Worked*	*Failed*
Image Classification	96	0
Visual Wake Words	95	10
Keyword Spotting	100	0
Anomaly Detection	96	0

Source: author

Among the total of 397 models tested, the success rate was of 97.48%. Expected potential causes for error could be insufficient ROM or RAM in the MCU, incompatibility of operators used by TFLM, device errors with the USB serial connection. However, all of the 10 models that failed, did so for the same reason: an error with the external tool "stm32tflm". When these models are given as input to this tensor arena estimation tool, it prints "HALTED" and hangs indefinitely. Looking at the hyperparameters of these 10 failed models, shown in Table 6.2, it is unclear as to what caused the stm32 tool to reject these models. Their ROM sizes are in the same range as the successful variations, so the ones that failed are neither smaller nor substantially larger than the others.

Table 6.2: Hyperparameters of the Visual Wake Words variations rejected by stm32tflm

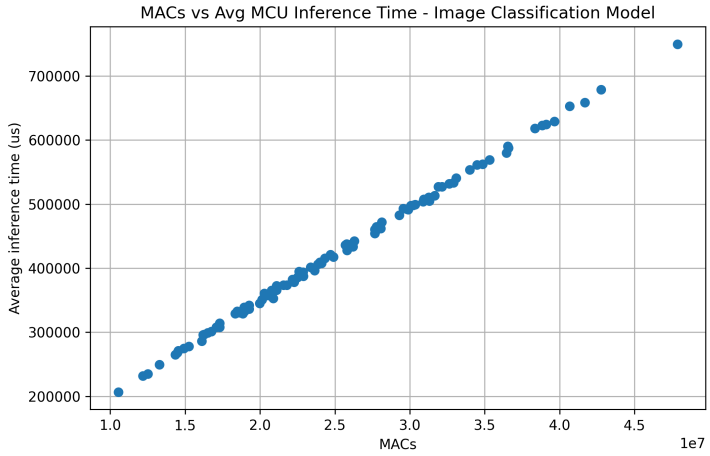
INITIAL_CHANNELS	*CHANNELS_MULTIPLIER*	*KERNEL*
10	1.2	3
10	2.0	2
10	2.0	3
10	2.0	4
4	1.4	2
4	1.4	3
5	1.2	3
5	1.8	2
6	1.4	2
8	2.0	2

Source: author

6.2 MCU Latency vs MACs

With the goal of assessing the correlation between number of MACs and real measured latency on the MCU, we plotted the graphs seen on Figures 6.1, 6.2, 6.3 and 6.4. The standard deviation of the 10 values averaged to produce each point was in the order of units, so small that it would not be visible as error bars in the graph, so it is omitted. As a way to quantify the correlation, the r coefficients for each model are shown on Table 6.3.

Figure 6.1: Correlation between number of MACs and measured inference time on Image Classification Model (source: author)



Analyzing the results, the four base benchmarking models can be clearly separated in two groups: two models that had almost perfect correlation (Image Classification and Anomaly Detection) and two that that high high but not perfect correlation (Visual Wake

Figure 6.2: Correlation between number of MACs and measured inference time on Visual Wake Words Model (source: author)

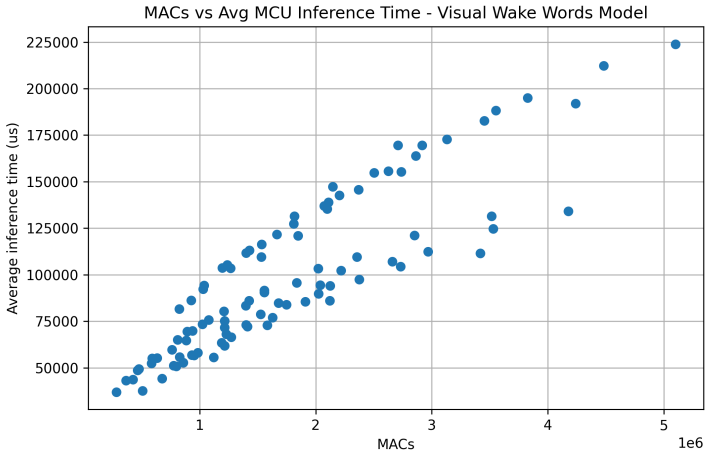
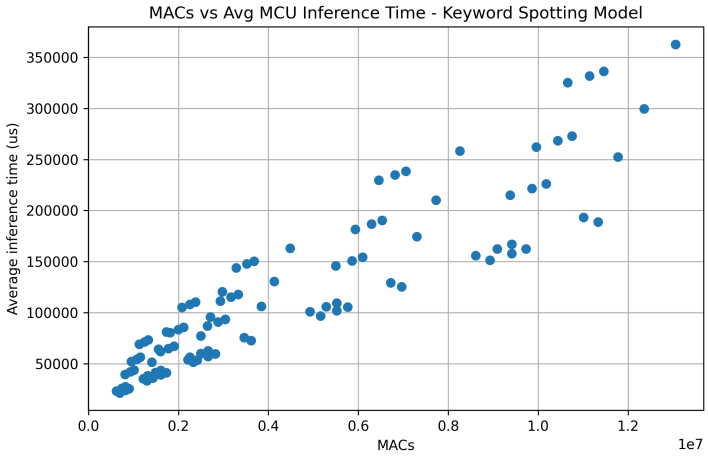


Figure 6.3: Correlation between number of MACs and measured inference time on Keyword Spotting Model (source: author)



Words and Anomaly Detection). In terms of neural architecture, the one feature that stands out that these last two have that the others do not is the presence of Depthwise Convolution layers. The Anomaly Detection model is the simplest one, with only Fully Connected layers, which possibly explains its perfect correlation. What is somewhat surprising is how the Image Classification model also belongs to the same group, despite having blocks with Convolution Layers, pooling and residual connections.

To illustrate a worst-case scenario for a Neural Architecture Search using MACs as latency estimation, we shine light on specific models that displayed the largest proportional difference between predicted and actual inference latency. For Keyword Spotting variations, let us consider Model A (CHANNELS=64, KERNEL=6, FIRST_KERNEL=10x4) which has 3.5M MACs and 147 ms of MCU inference time, and Model B (CHAN-

Figure 6.4: Correlation between number of MACs and measured inference time on Anomaly Detection Model (source: author)

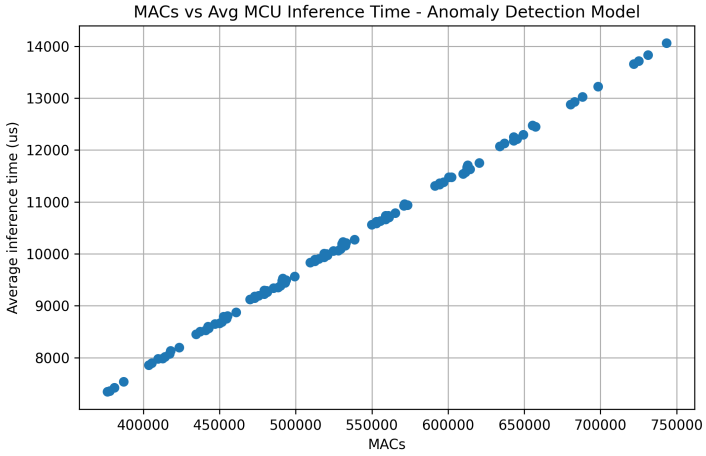


Table 6.3: Measured correlation between MACs and real latency

Base Model	*r coefficient*
Image Classification	0.9995
Visual Wake Words	0.8832
Keyword Spotting	0.9113
Anomaly Detection	0.9998

Source: author

NELS=128, KERNEL=3, FIRST_KERNEL=5x2) which has 8.9M MACs and 151 ms of MCU inference time. Just looking at number of MACs we would have estimated the latter to have around 2.5 times more inference time than the former and possibly discarded it from the search, when in fact it is only 1.02 times slower. A similar example can be also found in Visual Wake Words variations. When comparing Model C (INITIAL_CHANNELS=7, CHANNELS_MULTIPLIER=1.4, KERNEL=4) with 1.8M MACs and 131 ms of inference time, and Model D (INITIAL_CHANNELS=9, CHANNELS_MULTIPLIER=2.0, KERNEL=3) with 4.1M MACs and 134 ms of inference time, we would expect Model D to be 2.2 times slower than Model C, when on the MCU it is actually only 1.02 times slower.

6.3 MCU Latency vs Workstation Latency

In the same fashion as the previous section, the results for the experiment comparing workstation inference time with MCU inference time are on Figures 6.5, 6.6, 6.7 and

6.8. In this case, however, the standard deviations on the 10 measurements averaged for the Workstation CPU inference times were significant, which are therefore represented with the error bars on the x axis. The r coefficients for each model can be seen on Table 6.4.

Figure 6.5: Correlation between measured workstation CPU inference time and measured inference time on Image Classification Model (source: author)

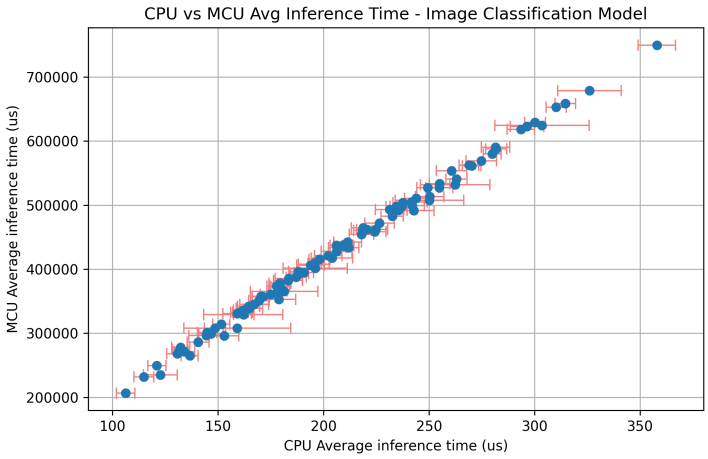
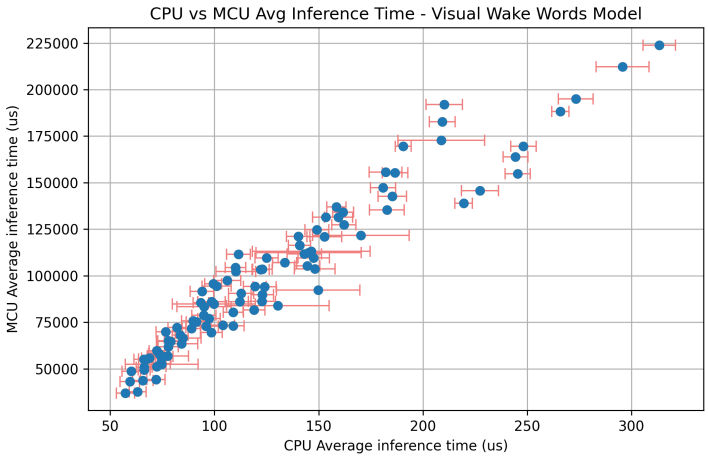


Figure 6.6: Correlation between measured workstation CPU inference time and measured inference time on Visual Wake Words Model (source: author)



Looking at the r coefficients for workstation inference, we can observe different patterns when compared to number of MACs. For this metric, only the Image Classification variations had almost perfect correlation, whereas the Anomaly Detection models, which previously belonged to the same cluster as Image Classification, now present a slightly smaller correlation. This could possibly be attributed to the notably high variance that all Anomaly Detection variations present for the workstation latency, visible in Figure

Figure 6.7: Correlation between measured workstation CPU inference time and measured inference time on Keyword Spotting Model (source: author)

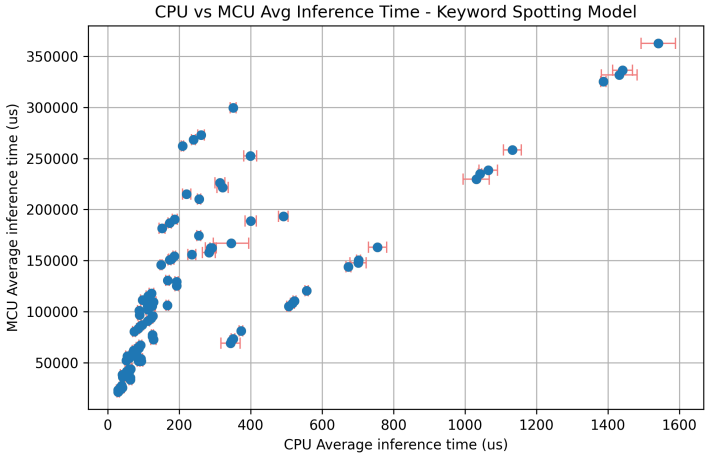
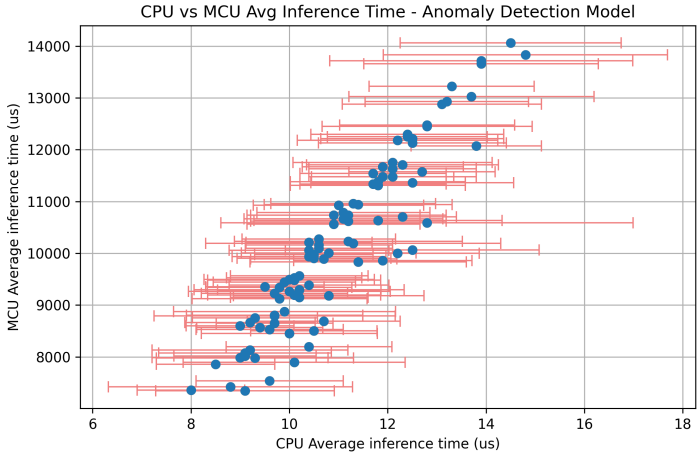


Figure 6.8: Correlation between measured workstation CPU inference time and measured inference time on Anomaly Detection Model (source: author)



6.8. The Visual Wake Words models also have very high r coefficients - higher than when using number of MACs. However, the case is different for Keyword Spotting. For these variations, the correlation is significantly lower than the others, and also lower than when using number of MACs. Analyzing the plot in Figure 6.7, there are two distinct classes of variation models, each one falling into one of the correlation lines. This separation is not seen on the plot for number of MACs (Figure 6.3), meaning that it reflects some particular bottleneck of the specific workstation setup.

Due to the fact that Keyword Spotting has the lowest correlation between workstation and MCU latency, we will focus on it for the worst-case scenario for NAS. A search process guided by workstation latency as metric would estimate Model E (CHANNELS=96, KERNEL=6, FIRST_KERNEL=20x8) with 1132 us workstation latency to

Table 6.4: Measured correlation between workstation latency and real MCU latency

Base Model	*r coefficient*
Image Classification	0.9983
Visual Wake Words	0.9616
Keyword Spotting	0.7547
Anomaly Detection	0.9337

Source: author

be 5.41 slower than Model F (CHANNELS=128, KERNEL=5, FIRST_KERNEL=5x2), which had 209 us of workstation latency. This choice would not at all reflect the actual target MCU inference times, where Model E (258K us MCU latency) is in fact 1.01 times faster than Model F (262K us MCU latency), instead of the assumed 5.41 times slower.

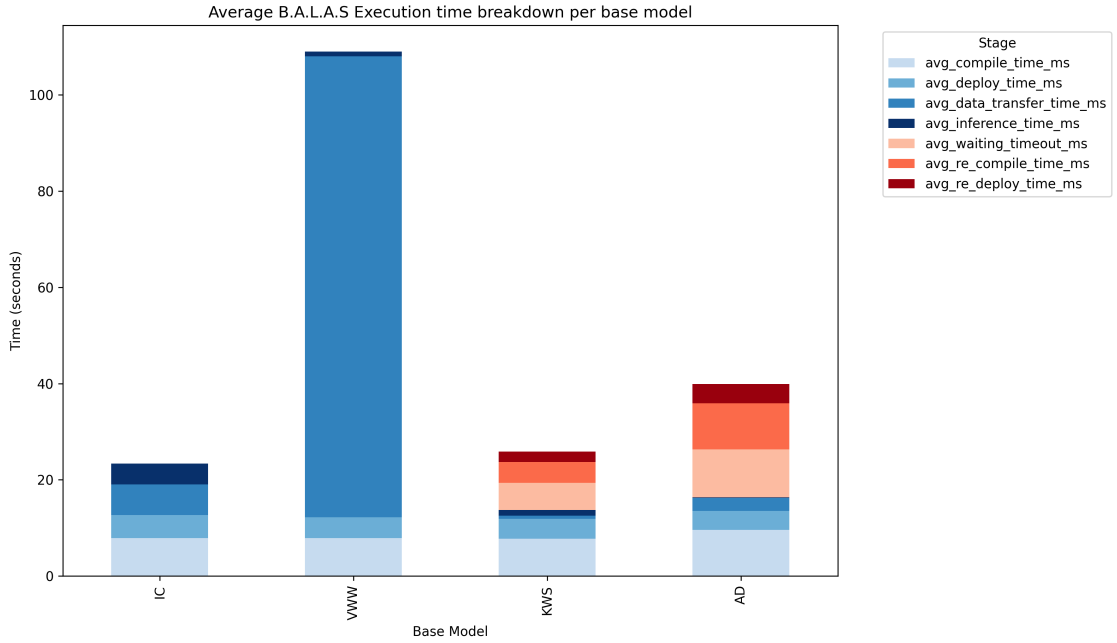
6.4 Timing

The execution time of each step of the B.A.L.A.S tool was measured, with the average times for profiling one model variation displayed in Figure 6.9. Some steps, such as arena_time, mac_time and cpp_codegen, had negligible execution times - in the order of units of milliseconds - so we opted to not show them on the graph, since they would be invisible and their labels would only take up space, distracting from the other relevant ones. Using the number of attempts needed, it was possible to deduce how much time was needed for waiting for the timeout, recompiling and redeploying, with these bars in red representing the time wasted due to underestimating the tensor arena size.

From Figure 6.9 we can clearly see how each base model's unique characteristics reflect on the execution times. The Visual Wake Words model has by far the largest input size (Table 4.1), which explains why the data transfer stage dominates the total time. The data transfer takes longer than inference for all models but Keyword Spotting, which, in contrast to Visual Wake Words, has the smallest input size. Given that the standard deviation of the MCU inference time was negligible in all models and variations, a clear optimization would be to run just one inference instead of averaging 10. This would reduce the inference stage time in 10 times, and, most importantly, would also reduce the data transfer stage in the same amount.

Another important aspect that can be seen in Figure 6.9 is how much the reattempts caused by tensor arena underestimations affected the total time. For both models where

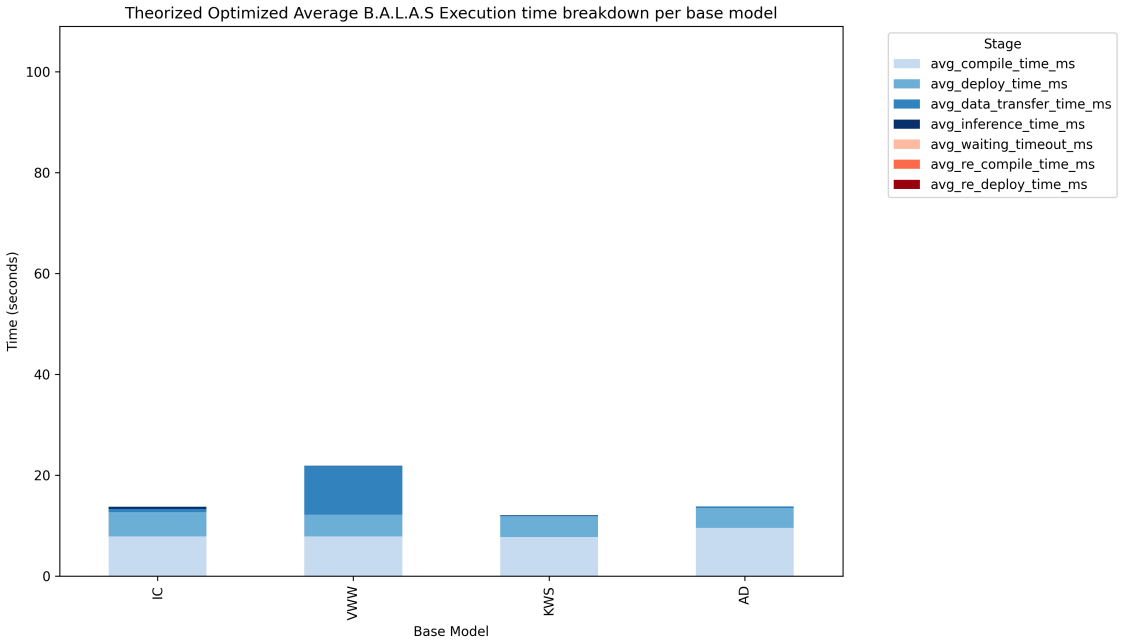
Figure 6.9: Execution times for each step of the automated profiling process (source: author)



this happened, Keyword Spotting and Anomaly Detection, the average time per variation profiled increased by 1.8 and 2.4 times respectively. No variation tested needed more than 3 attempts to reach the required tensor arena size, which means that if a constant of 10 KB had been added to the stm32tflm tool's estimation to compensate for its underestimations, all of the time wasted in reattempts would have been eliminated. However, this is no guarantee that reattempts would be eliminated for any model, only for the variations tested in this work.

Considering the optimizations proposed, it would be possible to have 10 times less execution time for inference and data transfer, and to eliminate the time wasted by reattempts. Figure 6.10 illustrates how the timing results would look with these optimizations. The experiments were not executed a second time with the optimizations implemented, rather it was simply calculated how the results would be with them. This shows that it is possible to profile inference times of real TinyML models on actual MCU hardware with an average of around 20 seconds per model.

Figure 6.10: Theoretical execution times for each step of the automated profiling process with optimizations (source: author)



7 CONCLUSION

This work presented the development of B.A.L.A.S, a tool that can be used for automated benchmarking of ANN latency on real microcontrollers, and that can be extended and adapted to support other MCUs by reimplementing the compilation and deployment stages, as well as the timer and UART functionalities. We demonstrated that B.A.L.A.S has high compatibility with variations of standard TinyML benchmarking models, even though the chosen stm32tflm tool fails on some models. Thus, finding another tensor arena estimation method that works on 100% of the tested models is a clear point of improvement for future research and development efforts.

Furthermore, the experiments showed that both number of MACs and workstation CPU inference time, when used as proxy metrics for real MCU inference latency, can be unreliable, displaying worst-case errors that could guide the search process to a suboptimal path. This highlights the importance of using a tool like B.A.L.A.S and measuring inference latency on-device when having low inference latency is critical for the target TinyML application. We also showed how, with proper optimizations to the process, it is possible to take in the order of 20 seconds per model to measure real hardware inference time, which is significantly less that the time it takes to train, even partially, a model. This puts in question the argument that on-device profiling takes too long, corroborating the work of (TAN et al., 2019) and sharing valuable timing data with the TinyML research community.

In light of these results, we argue for a workflow for TinyML latency-optimizing Neural Architecture Search processes. In the first stage, when the search space is still broad, fast latency proxies such as number of MACs and workstation inference time can be used to save some time, since they have a positive correlation with the real inference time. Then, in a second and final stage, when the search space is narrower, one can substitute the latency proxy with real automated measurements. This ensures that the search is not negatively affected by the unpredictable inaccuracies of the proxies, which are more apparent in narrower comparisons.

For future research expanding on this work, besides improving the tensor arena estimation compatibility, the same experiments and analysis could be made using the NPU accelerator of the microcontroller, exploring if this increases or decreases the quality of the proxy estimations. Moreover, another interesting research expanding on this work would be to use B.A.L.A.S and the proposed workflow to perform real search for

a TinyML task, comparing the final results of the model with searches performed just using proxies. This would serve as a clear example of how much performance gain can be expected from using this method for real TinyML tasks.

REFERENCES

ABADADE, Y. et al. A comprehensive survey on tinyml. **IEEE Access**, IEEE, v. 11, p. 96892–96922, 2023.

AL-SARAWI, S. et al. Internet of things market analysis forecasts, 2020–2030. In: IEEE. **2020 Fourth World Conference on smart trends in systems, security and sustainability (WorldS4)**. [S.l.], 2020. p. 449–453.

BACIU, V.-E.; STIENS, J.; SILVA, B. da. Mlino bench: A comprehensive benchmarking tool for evaluating ml models on edge devices. **Journal of Systems Architecture**, Elsevier, v. 155, p. 103262, 2024.

BANBURY, C. et al. Mlperf tiny benchmark. In: VANSCHOREN, J.; YEUNG, S. (Ed.). **Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks**. [s.n.], 2021. v. 1. Available from Internet: <https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/da4fb5c6e93e74d3df8527599fa62642-Paper-round1.pdf>.

BURRELLO, A. et al. Dory: Automatic end-to-end deployment of real-world dnns on low-cost iot mcus. **IEEE Transactions on Computers**, IEEE, v. 70, n. 8, p. 1253–1268, 2021.

BURRELLO, A. et al. Enhancing neural architecture search with multiple hardware constraints for deep learning model deployment on tiny iot devices. **IEEE Transactions on Emerging Topics in Computing**, IEEE, v. 12, n. 3, p. 780–794, 2023.

CAI, H.; ZHU, L.; HAN, S. ProxylessNAS: Direct neural architecture search on target task and hardware. In: **International Conference on Learning Representations**. [s.n.], 2019. Available from Internet: <https://arxiv.org/pdf/1812.00332.pdf>.

CHEN, G.; PARADA, C.; HEIGOLD, G. Small-footprint keyword spotting using deep neural networks. In: IEEE. **2014 IEEE international conference on acoustics, speech and signal processing (ICASSP)**. [S.l.], 2014. p. 4087–4091.

CROCIONI, G. et al. Li-ion batteries parameter estimation with tiny neural networks embedded on intelligent iot microcontrollers. **IEEE Access**, IEEE, v. 8, p. 122135–122146, 2020.

DAVID, R. et al. Tensorflow lite micro: Embedded machine learning for tinyml systems. **Proceedings of Machine Learning and Systems**, v. 3, p. 800–811, 2021.

DUTTA, L.; BHARALI, S. Tinyml meets iot: A comprehensive survey. **Internet of Things**, Elsevier, v. 16, p. 100461, 2021.

ESSA, M. E.-S. M. et al. Low-cost hardware in the loop for intelligent neural predictive control of hybrid electric vehicle. **Electronics**, MDPI, v. 12, n. 4, p. 971, 2023.

FARAONE, A.; DELGADO-GONZALO, R. Convolutional-recurrent neural networks on low-power wearable platforms for cardiac arrhythmia detection. In: IEEE. **2020 2nd IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)**. [S.l.], 2020. p. 153–157.

GAL-ON, S.; LEVY, M. Exploring coremark a benchmark maximizing simplicity and efficacy. **The Embedded Microprocessor Benchmark Consortium**, v. 6, n. 23, p. 87, 2012.

GHANATHE, N. P. **Towards efficient and intelligent tinyML: acceleration, architectures, and monitoring**. Thesis (PhD) — University of British Columbia, 2025.

GUO, Y. et al. Depthwise convolution is all you need for learning multiple visual domains. In: **Proceedings of the AAAI Conference on Artificial Intelligence**. [S.l.: s.n.], 2019. v. 33, n. 01, p. 8368–8375.

HAN, H.; SIEBERT, J. Tinyml: A systematic review and synthesis of existing research. In: IEEE. **2022 International Conference on Artificial Intelligence in Information and Communication (ICAIIC)**. [S.l.], 2022. p. 269–274.

HE, K. et al. Deep residual learning for image recognition. In: **Proceedings of the IEEE conference on computer vision and pattern recognition**. [S.l.: s.n.], 2016. p. 770–778.

HUANG, Z. et al. U-toe: universal tinyml on-board evaluation toolkit for low-power iot. In: IEEE. **2023 12th IFIP/IEEE International Conference on Performance Evaluation and Modeling in Wired and Wireless Networks (PEMWN)**. [S.l.], 2023. p. 1–6.

KEMPEN, P. van et al. Mlonmcu: Tinyml benchmarking with fast retargeting. In: **Proceedings of the 2023 Workshop on Compilers, Deployment, and Tooling for Edge AI**. [S.l.: s.n.], 2023. p. 32–36.

KRASTEVA, V.; STOYANOV, T.; JEKOVA, I. Implementing deep neural networks on arm-based microcontrollers: Application for ventricular fibrillation detection. **Applied Sciences**, MDPI, v. 15, n. 4, p. 1965, 2025.

LIBERIS, E.; DUDZIAK, Ł.; LANE, N. D. μnas: Constrained neural architecture search for microcontrollers. In: **Proceedings of the 1st Workshop on Machine Learning and Systems**. [S.l.: s.n.], 2021. p. 70–79.

MOUHA, R. A. R. A. et al. Internet of things (iot). **Journal of Data Analysis and Information Processing**, Scientific Research Publishing, v. 9, n. 02, p. 77, 2021.

NXP. 2025. <https://www.nxp.com/products/MCX-N94-N54-N53-N52-N24>. Accessed: 25 Nov. 2025.

ORĂŞAN, I. L.; SEICULESCU, C.; CALEANU, C. D. Benchmarking tensorflow lite quantization algorithms for deep neural networks. In: IEEE. **2022 IEEE 16th International Symposium on Applied Computational Intelligence and Informatics (SACI)**. [S.l.], 2022. p. 000221–000226.

PEREGO, R. et al. Autotinyml for microcontrollers: Dealing with black-box deployability. **Expert Systems with Applications**, Elsevier, v. 207, p. 117876, 2022.

RAWAT, P. et al. A comprehensive study based on mfcc and spectrogram for audio classification. **Journal of Information and Optimization Sciences**, v. 44, n. 6, p. 1057–1074, 2023.

REDDI, V. J. et al. Mlperf inference benchmark. In: IEEE. **2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)**. [S.l.], 2020. p. 446–459.

SAHA, S. S.; SANDHA, S. S.; SRIVASTAVA, M. Machine learning for microcontroller-class hardware: A review. **IEEE Sensors Journal**, IEEE, v. 22, n. 22, p. 21362–21390, 2022.

SZE, V. et al. Efficient processing of deep neural networks: A tutorial and survey. **Proceedings of the IEEE**, Ieee, v. 105, n. 12, p. 2295–2329, 2017.

TAN, M. et al. Mnasnet: Platform-aware neural architecture search for mobile. In: **Proceedings of the IEEE/CVF conference on computer vision and pattern recognition**. [S.l.: s.n.], 2019. p. 2820–2828.

TORELLI, P.; BANGALE, M. Measuring inference performance of machine-learning frameworks on edge-class devices with the mlmark benchmark. **Techincal Report. Available online: https://www. eembc. org/techlit/articles/MLMARK-WHITEPAPERFINAL-1. pdf (accessed on 5 April 2021)**, 2021.

USTUBIOGLU, A.; USTUBIOGLU, B.; ULUTAS, G. Mel spectrogram-based audio forgery detection using cnn. **Signal, Image and Video Processing**, Springer, v. 17, n. 5, p. 2211–2219, 2023.

ZOPH, B. et al. Learning transferable architectures for scalable image recognition. In: **Proceedings of the IEEE conference on computer vision and pattern recognition**. [S.l.: s.n.], 2018. p. 8697–8710.